

A Lightweight Blockchain-Assisted Certificateless Cloud Data Integrity Auditing Scheme Without Third-Party Auditor

Chen Zhu^{1b}, Yang Lu^{2b}, Nian Xia^{3b}, *Member, IEEE*, Jiguo Li^{4b}, and Yinxia Sun^{5b}

Abstract—Data Integrity Auditing (DIA) enables users to remotely verify whether their data saved in third-party clouds has been maliciously tampered with or compromised. As an extension of DIA in certificateless cryptography, certificateless DIA (CL-DIA) integrates the merits of conventional public-key cryptography (no key escrow) and identity-based cryptography (no certificates). However, CL-DIA schemes depend on a reliable third-party auditor (TPA) to perform integrity audits, inevitably suffering from performance bottleneck and single-point failure problems. Moreover, almost all current CL-DIA schemes were designed with computationally expensive bilinear pairings. Cryptanalysis demonstrates that the existing unique pairing-free CL-DIA scheme fails to achieve the unforgeable security of auditing proofs. In this work, we put forward a lightweight blockchain-assisted CL-DIA scheme. The scheme achieves DIA through the blockchain instead of a single TPA, thereby overcoming the problems caused by the TPA-based centralized auditing model. Then, by avoiding time-consuming pairing operations and employing edge servers in generating verifiable tags for the uploaded data of users, its performance surpasses previous pairing-based CL-DIA schemes, particularly in terms of computation efficiency. Furthermore, we provide formal proofs in the random oracle model demonstrating that our scheme achieves unforgeability of verifiable tags and auditing proofs, ensures data privacy security, and is resistant to collusion attacks between the EN and the CSP. Finally, experimental results show that when auditing 25 file blocks, our scheme only costs 0.29s, which reduces the total time cost of integrity auditing phase by 48.2%-85.5% compared to current pairing-based CL-DIA schemes.

Index Terms—Integrity auditing, certificateless, lightweight, blockchain-assisted, TPA free.

I. INTRODUCTION

CLOUD storage is a concept derived from cloud computing, leveraging cloud computing technology and

architecture to offer users on-demand, pay-as-you-go storage solutions. Cloud storage reduces expenses related to data management, hardware procurement, and maintenance, thereby leading more users to opt for cloud storage to save their data. Once users have moved data to the cloud, they may not keep local copies and cannot manage the data as they would locally. This may lead to three potential scenarios of data integrity compromises: (1) software failure or hardware damage resulting in data loss; (2) data being maliciously tampered with by hackers or other users; (3) cloud service providers (CSPs) possibly deleting infrequently accessed data without authorization for economic reasons. In light of these risks, the healthcare industry is increasingly embracing cloud infrastructure. Take the adoption of standardized cloud-based health data platforms as an example. By leveraging interoperability standards such as FHIR, HL7, and DICOM, healthcare providers can store and manage electronic medical records (EMRs) in the cloud through services like AWS HealthLake and Google Cloud Healthcare API [1]. This approach reduces the cost of maintaining on-premise data centers while improving data sharing efficiency across institutions. However, once data is stored remotely, it remains vulnerable to tampering or transmission errors. A compromised record - such as an altered medication list or falsified lab result - could mislead clinicians and lead to serious medical errors. Therefore, there is a need for an effective mechanism that enables users to verify the integrity of their data stored in the cloud.

Data integrity auditing (DIA) technology provides an effective solution to addressing the above issue. Through DIA, data owners can generate an unforgeable and verifiable tag for each file block and send these tags along with their data files to the CSP. Upon receiving an integrity challenge (a random index list of file blocks) initiated by the verifier (either a data owner or a TPA), the CSP should use the challenge-designated file blocks and their corresponding tags to create an auditing proof, which is then returned to the verifier for validation. In this way, data owners are able to determine if their cloud data maintains integrity. Since its introduction, many DIA schemes [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14] have been proposed. However, most of them are on the basis of conventional public-key cryptography which has certificate management issues, or identity-based cryptography which has escrow issues.

As a novel public key cryptographic primitive, certificateless cryptography (CLC) [15] combines the strong points of conventional public-key cryptography (no escrow) and identity-based cryptography (no certificates). Therefore, com-

Received 11 April 2025; revised 19 July 2025 and 31 August 2025; accepted 2 January 2026. Date of publication 12 January 2026; date of current version 15 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 61772009, Grant 61972095, and Grant 62072104; and in part by the Open Research Fund of Key Laboratory of Analytical Mathematics and Applications (Fujian Normal University), Ministry of Education, China. The associate editor coordinating the review of this article and approving it for publication was Dr. Sushmita Ruj. (Corresponding author: Yang Lu.)

Chen Zhu, Nian Xia, and Yinxia Sun are with the School of Computer and Electronic Information, Nanjing Normal University, Nanjing 210023, China (e-mail: zc05042@163.com; nian.xia@nnu.edu.cn; bela_suno@163.com).

Yang Lu is with the School of Computer and Electronic Information, Nanjing Normal University, Nanjing 210023, China, and also with the State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing 210023, China (e-mail: luyangnsd@163.com).

Jiguo Li is with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou 350117, China (e-mail: ljg1688@163.com).

Digital Object Identifier 10.1109/TIFS.2026.3652010

pared to those DIA schemes over conventional public-key cryptography or identity-based cryptography, certificateless data integrity auditing (CL-DIA) has obvious advantages. In recent years, the design of CL-DIA schemes has garnered significant attention from the cryptographic community [16], [17], [18], [19], [20], [21], [22], [23], [24].

A. Our Motivations

So far, many CL-DIA schemes have been proposed. However, there are still several issues to be resolved.

- **Centralized auditing mode.** Current CL-DIA schemes rely on a TPA to achieve DIA. For the most part, the TPA is assumed to be entirely trustworthy. In reality, it might collude with the CSP to conceal data integrity compromises for business purposes. This is possible because the TPA might be bribed or maliciously hijacked. Besides, the single TPA-based DIA has performance bottleneck and single-point failure problems. Clearly, if the TPA fails, the system cannot run normally, or if the TPA has to process DIA requests from numerous users, it inevitably suffers from poor performance, leading to severe system delays. Furthermore, as discussed in [25], the challenge messages in TPA-based DIA schemes are created by the TPA. During the proof verification, the TPA may expose the challenged file blocks via the auditing proofs, thereby leading to the data privacy leakage.
- **Deficient security model definition.** A CL-DIA scheme should simultaneously provide the existential unforgeability of verifiable tags under the chosen message attack (Tag-EUF-CMA) and the existential unforgeability of auditing proofs under the chosen message attack (Proof-EUF-CMA). However, the security model for the Proof-EUF-CMA security adopted by previous CL-DIA schemes [17], [18], [21], [22], [26] appears unable to capture attack scenarios in which the CSP forges the auditing proofs for the purposely deleted or accidentally-damaged file blocks using the original verifiable tags. Cryptanalysis shows that the lightweight CL-DIA scheme proposed by Zhao et al. [26] is vulnerable to such auditing proof forging attacks, although it was proven secure under the existing Proof-EUF-CMA security model. Specifically, in Zhao et al.'s scheme, an auditing proof $Proof = (\zeta, \xi, \delta)$ for a challenge $Chal = (ID, Fname, \{(i, v_i)\}_{i \in I})$ is created as follows:

$$\begin{aligned}\zeta &= \sum_{i \in I} v_i H_5(ID, m_i, U_i, h_{4i})(R + h_2 P_{pub}), \\ \xi &= \sum_{i \in I} v_i U_i \sum H_5(ID, m_i, U_i, h_{4i}), \\ \delta &= \sum_{i \in I} v_i \sigma_i,\end{aligned}$$

where ID is a DO's identity, $Fname$ is the file-name to be audited. The set $\{(i, v_i)\}_{i \in I}$ is comprised of file block indexes and corresponding random numbers, (U_i, σ_i) is the verifiable tag of the file block m_i , $h_{4i} = H_4(ID, Fname, i, PK)$ and $h_2 = H_2(ID, P_{pub}, PK)$ are two hash values and $PK = (X, R)$ is the DO's public

key. Clearly, the CSP can generate valid auditing proofs only using verifiable tags and hash values related to the challenged file blocks. This implies that even if the CSP has deleted the file blocks and only retains the corresponding hash values and verifiable tags, it can still create valid auditing proofs.

- **High computation overhead.** Almost all constructions of current CL-DIA schemes depend on bilinear pairings, which are computationally expensive. Therefore, these schemes inevitably incur high computation costs to generate verifiable tags (on user terminals) and conduct auditing proof verification (on the TPA). This not only puts high demands on the performance of user terminals but also reduces the auditing efficiency of the system. Although Zhao et al. [26] recently proposed a lightweight CL-DIA scheme without bilinear pairings, their scheme fails to achieve the Proof-EUF-CMA security. Until now, building a pairing-free CL-DIA scheme that satisfies both the Tag-EUF-CMA and Proof-EUF-CMA security remains an open challenge.

B. Our Contributions

To address the aforementioned issues, we put forward a lightweight blockchain-assisted CL-DIA (LB-CL-DIA) scheme. As shown in TABLE I, the proposed scheme utilizes blockchain and edge node collaboration to achieve data auditing instead of relying on a single TPA. This scheme also does not rely on computationally expensive bilinear pairings. Moreover, it is formally proved to satisfy the existential unforgeability of both verifiable tags and auditing proofs. Specifically, our main contributions are summarized below:

- **Blockchain-assisted DIA.** We propose a blockchain-assisted CL-DIA scheme that performs integrity verification without TPA. Blockchain is a trusted data management mechanism built on a distributed network. It relies on cryptography, decentralized consensus protocols, and smart contracts to guarantee the immutability, transparency, and traceability of information. There are no centralization problems, such as collusion or single-point failures, in the blockchain. In our scheme, we employ smart contracts to generate challenge messages and verify the auditing proofs provided by the CSP. By integrating the blockchain into DIA, our scheme effectively addresses the problems inherent in the current TPA-based CL-DIA schemes.
- **Enhanced security model.** The main weakness of the previous Proof-EUF-CMA security model is that no restrictions are imposed on the adversary's tag queries. It is unreasonable to make an adversary query the verifiable tags for the purposely deleted or accidentally damaged file blocks. We present an enhanced Proof-EUF-CMA security model for LB-CL-DIA. The model prohibits the adversary from querying the verifiable tag for any corrupted or missing file block. Thus, it ensures that the CSP can produce valid proofs only with both file blocks and corresponding verifiable tags. In addition, we have considered the security of data privacy protection, that is,

TABLE I
COMPARISON BETWEEN OUR SCHEME AND CURRENT DIA SCHEMES

Schemes	Lightweight	TPA Free	Data Privacy	Tag-EUF-CMA	Proof-EUF-CMA	Experimental efficiency	
						Computation cost (s)	Proof length (bits)
[17]	✗	✗	✗	✓	✓	2.01	2688
[18]	✗	✗	✓	✓	✓	1.02	1152
[21]	✗	✗	✗	✓	✓	0.98	672
[22]	✗	✗	✓	✓	✓	0.56	1152
[26]	✓	✗	✓	✓	✗	0.31	800
[27]	✗	✗	✓	✓	✓	1.41	1152
Ours	✓	✓	✓	✓	✓*	0.29	960

The computation cost in the table is the total time cost during the integrity auditing phase to audit 25 file blocks; the number of sectors involved in the proof length is 4 for reference.

✗: not satisfied; ✓: satisfied; ✓*: satisfied in our enhanced security model.

even if an adversary gains a file's tags or audit proofs, he cannot derive any information about the message blocks.

- **Lightweight construction.** The construction of our scheme is over the elliptic curve and does not rely on costly pairings. Moreover, it introduces edge servers to assist users in generating verifiable tags for uploaded files and periodically initiating audit requests to the blockchain. As shown in TABLE I, our experiments (see Section VI for details) indicate that, compared to existing pairing-based CL-DIA schemes [17], [18], [21], [22], our scheme reduces the time cost of integrity auditing phase by 48.2%-85.5%. It also offers comparable efficiency against the current pairing-free CL-DIA scheme [26]. Therefore, our scheme is very suitable for resource-limited user terminals. As far as we know, it is currently the sole lightweight CL-DIA scheme that satisfies both the Tag-EUF-CMA and Proof-EUF-CMA security.

C. Related Works

In 2004, Deswarte et al. [28] presented the first remote DIA scheme which is based on conventional public-key cryptography. However, the scheme requires the retrieval of all remote files, which incurs significant communication, computation and storage overheads. In 2007, Ateniese et al. [2] introduced the notion of provable data possession (PDP) which pioneers the use of sampling-based methods to audit the integrity of cloud data. Their scheme is constructed on the basis of RSA signature, which can effectively detect whether a server has stored data intact. Erway et al. [3] presented a DIA scheme that supports dynamic operations, which is based on the work of Ateniese et al. [2]. To alleviate users' heavy computational burden, Wang et al. [4] introduced the TPA-based auditing model into integrity auditing. Subsequently, an integrity auditing scheme supporting privacy preservation was proposed by Wang et al. [5], which ensures that the TPA cannot leak the auditing information of cloud data. Since then, a considerable number of TPA-based DIA schemes have been introduced and thoroughly researched from different aspects to support various application requirements, such as key updating [6], user revocation [7], [8], privacy protection [9], [10], group sharing [11], [12], data dynamics [29], designated verifier [30], and so on. However, all the above-mentioned schemes were designed

using conventional public-key cryptography. To address the certificate management issue, some works (e.g., [13], [14]) introduced the idea of identity-based cryptography into DIA to propose identity-based DIA schemes. Li et al. [31] proposed the first identity-based PDP protocol for multiple copies on distributed cloud storage servers. Subsequently, Li et al. [32] also proposed a DIA scheme that can eliminate certificate management problems. However, because of the flaw originating from identity-based cryptography, these schemes suffer from the escrow issue. He et al. [16] combined certificateless cryptography with DIA and first proposed the CL-DIA model. Subsequently, several CL-DIA schemes have been proposed [17], [18], [19], [20], [21], [22], [23], [24], but they all utilize bilinear pairings which incur high computation costs. Recently, Zhao et al. [26] presented a pairing-free CL-DIA scheme for cloud-assisted medical cyber-physical systems.

Meanwhile, many researchers have also begun exploring the integration of blockchain with integrity auditing technology to address issues arising from centralized management. Xue et al. [33] introduced an identity-based DIA scheme based on blockchain, achieving resistance against procrastinating TPAs. Zhang et al. [34] presented a blockchain-based CL-DIA scheme to resist malicious TPAs. Lin et al. [35] proposed a DIA scheme based on a consortium blockchain. In their scheme, the authorized parties can audit the auditor's behavior recorded in the consortium blockchain to verify the correctness of the results. Wang et al. [36] proposed a DIA scheme that allows the TPA and the CSP to complete DIA without direct interaction. They also established a reliable payment protocol using smart contracts. Banerjee et al. [25] identified a data privacy leakage vulnerability in numerous existing TPA-based DIA schemes and introduced a countermeasure to fix the security flaw. They also designed a protocol that leverages smart contracts and state channel technology to ensure system security and provide a practical audit solution. Sengupta et al. [37] implemented a fair and secure data sharing scheme using blockchain, and they validated the integrity of the received data based on smart contracts. Besides, there are also several works [27], [38], [39], [40], [41], [42], [43] that explore the combination of blockchain and DIA. Although the above schemes integrated the blockchain into the DIA process, they still rely on the TPA to complete integrity verification

TABLE II
NOTATIONS

Notations	Definitions
F_p	A finite field with p order
E/F_p	An elliptic curve over F_p
$params$	The system public parameters
msk	The master secret key
$\mathbb{G}, q, P$	The elliptic curve parameters
H_1, H_2, H_3	The hash functions
PSK	The DO's partial secret key
PK, SK	The DO's public key and secret key
$Fname$	The filename
m_i	The i -th file block
σ_i	The verifiable tag of m_i
$Chal$	The BN's auditing challenge
Ω	The DO's storage data
$Proof$	The CSP's auditing proof
θ	The auditing result
$Account_{DO}$	The DO's wallet address
$Account_{CSP}$	The CSP's wallet address
Pay_{store}	The DO's storage fee
Pay_{Comp}	The CSP's compensation fee

and therefore suffer from the related issues caused by the centralized auditing mode.

D. Organization

The paper is structured as follows. In Section II, we introduce the preliminaries of this paper, including the relevant notations and mathematical primitives used in this paper. Section III gives the system model, scheme definition, and security definitions. The specific construction and workflow of our scheme are given in Section IV. We provide the security analysis and efficiency evaluation of our scheme in V and Section VI, respectively. Finally, we conclude this paper in Section VII.

II. PRELIMINARIES

A. Definitions

Table II lists the notations used throughout this paper.

B. Elliptic Curve Group

Let F_p be a finite field, where p is a prime. An elliptic curve E/F_p defined over F_p is characterized by an equation $y^2 = x^3 + ax + b$ satisfying $4a^3 + 27b^2 \neq 0 \pmod{p}$, where $a, b \in F_p$. The elliptic curve group can be formally described as $\mathbb{G} = \{(x, y) \in F_p \times F_p : y^2 = x^3 + ax + b \mid 4a^3 + 27b^2 \neq 0 \pmod{p} \wedge a, b \in F_p\} \cup \mathcal{O}$, where $\mathcal{O}$ is the infinite point. Let q denote the order of $\mathbb{G}$ and $\mathcal{P}$ be a generator of $\mathbb{G}$. There are two fundamental operations over $\mathbb{G}$:

- Point addition: Let Q_1 and Q_2 be two random points in $\mathbb{G}$. Ln represents the straight line connecting Q_1 and Q_2 (or the tangent to E/F_p if Q_1 equals Q_2). Then $Q_3 = Q_1 + Q_2$, where Q_3 corresponds to the point of symmetry of the intersection between Ln and E/F_p about the x-axis.
- Scalar multiplication: Scalar multiplication is the operation of repeatedly adding a point $\mathcal{P}$ to itself, denoted as $t\mathcal{P} = \mathcal{P} + \mathcal{P} + \mathcal{P} + \dots + \mathcal{P}$, where $t \in \mathbb{Z}_q^*$.

C. Hardness Assumption

The security of our LB-CL-DIA scheme relies on the difficult assumption of the elliptic curve discrete logarithm problem (ECDLP).

Definition 1: Given an elliptic curve group $\mathbb{G}$ with order q and generator $\mathcal{P}$, and $Q \in \mathbb{G}$. The ECDLP assumption is that the probability of calculating an $x \in \mathbb{Z}_q^*$ in polynomial time such that $Q = x\mathcal{P}$ is negligible.

III. SCHEME AND SECURITY DEFINITIONS

A. System Model

The LB-CL-DIA system model comprises five distinct kinds of entities.

- Key Generation Center (KGC): The KGC is responsible for system initialization and the creation of a partial secret key for the data owner based on his/her identity. KGC is semi-trusted in CLC [15] and is not involved in the auditing process.
- Data Owner (DO): The DO identified by ID sends his/her data files (split into several blocks) to the CSP for storage. In order to ensure the integrity of the data, the DO uses his/her secret key to create a verifiable tag for each file block with the assistance of the adjacent edge node.
- The Cloud Service Provider (CSP): The CSP possesses mighty computing and storage capacities. It is responsible for data storage and dealing with DIA challenges emanating from the blockchain network at any time.
- Edge Node (EN): The EN is a computationally powerful edge server that can effectively reduce the computation burden on the DO. The EN can assist the DO in computing the most time-consuming part of the tag generation process. Moreover, it is also responsible for periodically initiating DIA requests to the BN and sending the returned auditing results to the DO.
- Blockchain Network (BN): The BN handles bookkeeping (including the metadata of the DO's cloud data and the auditing results) and completes DIA. Upon receiving a DIA request from the EN, it executes a smart contract to generate a challenge message which is subsequently transmitted to the CSP. Later, the BN validates the proof returned from the CSP and forwards the auditing result to the EN.

As shown in the Fig. 1, the KGC initially creates a master secret key and a set of public parameters. It then transmits the partial secret key (PSK) to the DO in Step ①. In Step ②, the DO generates his own key pair PK/SK. Steps ③-⑤ describe the tag generation process: The DO first sends a partial tag request to the EN, and then generates a verifiable tag σ after receiving the partial tag from the EN. Subsequently, in Step ⑥, the DO forwards the data to the CSP. Upon receipt, the CSP verifies the integrity of the data, stores it securely, and returns the storage result to the DO in Step ⑦. In Step ⑧, the DO submits metadata of the stored data to the BN. Steps ⑨-⑫ depict a complete audit process. The EN initiates an auditing request. In response, the BN issues a challenge $chal$ to the CSP, which is then verified through a proof $proof$. The auditing result is subsequently relayed back to the EN.

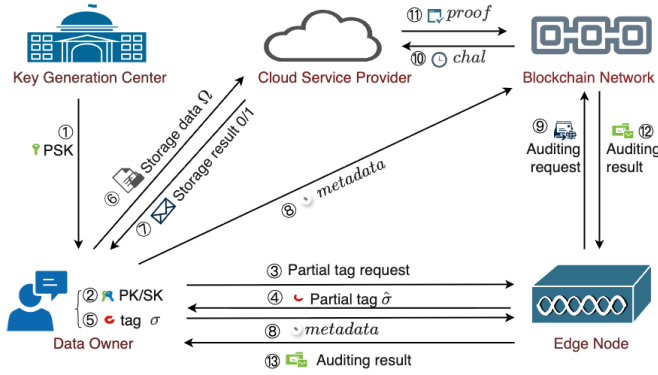


Fig. 1. System model of LB-CL-DIA.

Finally, in Step ⑬, the EN communicates the auditing result to the DO, thereby ensuring the integrity and security of the stored data.

B. Scheme Definition

Formally, a LB-CL-DIA scheme comprises of nine algorithms {**Setup**, **PartialKeyGen**, **UserKeyGen**, **TagGen**, **TagVerify**, **Store**, **ChalGen**, **ProofGen**, **ProofVerify**} such that:

- **Setup**(k): Given a security parameter k , the KGC executes the algorithm to produce a master secret key msk and a set of public parameters $params$.
- **PartialKeyGen**($params, msk, ID$): Given $params$, msk and the DO's identifier ID , the KGC executes the algorithm to produce the partial secret key PSK for DO.
- **UserKeyGen**($params, PSK$): Given $params$ and the partial secret key PSK , the DO performs this algorithm to create a pair of public and secret keys (PK, SK).
- **TagGen**($params, ID, SK, m_i, i, Fname$): Given $params$, the identifier ID and secret key SK of the DO, and a file block m_i along with its index i and filename $Fname$, the DO and the EN collaboratively execute this algorithm to produce a tag σ_i for m_i .
- **TagVerify**($params, ID, PK, m_i, i, Fname, \sigma_i$): Given $params$, the identifier ID and public key PK of the DO, a file block m_i along with its index i and filename $Fname$, and a verifiable tag σ_i , the CSP performs this algorithm to verify the validity of σ_i . The output of the algorithm is 1 or 0 which indicate that σ_i is a valid or invalid verifiable tag for m_i .
- **Store**($params, ID, SK, F, Fname$): Given $params$, the DO's identifier ID and secret key SK , and a data file F with filename $Fname$, the DO and the CSP execute this algorithm jointly to agree on the uploaded data. During the data store process, the DO runs **TagGen** to generate the block tags for F , and the CSP runs **TagVerify** to verify the validation of tags. The output of the algorithm is 1 or 0 indicating the acceptance or rejection of the uploaded data.
- **ChalGen**($params, ID, Fname, n$): Given $params$, the DO's identifier ID , a filename $Fname$ and the number n of file blocks, the BN performs the algorithm to create

an auditing challenge message $Chal$ after receiving an auditing request from the EN.

- **ProofGen**($params, Chal, \Omega$): Given $params$, an auditing challenge message $Chal$ and the DO's storage data Ω , the CSP executes this algorithm to create the response auditing proof $Proof$.
- **ProofVerify**($params, Chal, Proof, PK$): Given $params$, an auditing challenge message $Chal$, an auditing proof $Proof$ and the DO's public key PK , this algorithm outputs the auditing result 1/0 indicating that data integrity is guaranteed or not.

C. Security Definitions

An LB-CL-DIA scheme should satisfy the following security:

- Existential unforgeability of verifiable tags under the chosen message attack (Tag-EUF-CMA): This security guarantees that an adversary is unable to forge the verifiable tag on any file block.
- Existential unforgeability of auditing proofs under the chosen message attack (Proof-EUF-CMA): This security guarantees that an adversary is unable to forge the auditing proof on any given challenge.
- Data privacy security: This security guarantees that an adversary is unable to extract valid information from given auditing proofs.

To define these three security notions, four types of adversaries are taken into consideration:

- Type 1 (denoted by $\mathcal{A}_1$) represents an external attacker (including the CSP) who has the ability to substitute public keys and attempts to forge the verifiable tags of file blocks.
- Type 2 (denoted by $\mathcal{A}_2$) represents the KGC who refrains from replacing public keys and attempts to forge the verifiable tags of file blocks.
- Type 3 (denoted by $\mathcal{A}_3$) represents the CSP who has the ability to substitute public keys and attempts to forge the auditing proof on any given challenge.
- Type 4 (denoted by $\mathcal{A}_4$) represents an external attacker (excluding the CSP) who is allowed to query the private key for any identifier and attempts to extract valid information from given auditing proofs.

Tag-EUF-CMA Security. This security is formalized through the following two adversarial games:

Tag-EUF-CMA Game-I. This game is played by the challenger $\mathcal{C}$ and $\mathcal{A}_1$.

Setup Phase. $\mathcal{C}$ runs the **Setup** algorithm to create $params$ and msk , and gives $params$ to $\mathcal{A}_1$.

Query Phase. $\mathcal{A}_1$ makes queries to the following oracles.

- 1) $O^{CreateUser}$: On receiving an identifier ID , $\mathcal{C}$ provides a public key PK as a response. If ID has not been recorded, $\mathcal{C}$ runs the **PartialKeyGen** and **UserKeyGen** algorithms to create (PSK, SK, PK) for ID and returns PK . It is assumed that the rest of the oracles in the game merely respond to the recorded identifiers.
- 2) $O^{PartialKey}$: On receiving an identifier ID , $\mathcal{C}$ returns a partial secret key PSK .

- 3) $O^{PrivateKey}$: On receiving an identifier ID , $\mathcal{C}$ returns a secret key SK .
- 4) $O^{ReplacePublicKey}$: On receiving an identifier ID and a fake public key PK' , $\mathcal{C}$ substitutes the public key corresponding to ID with PK' .
- 5) O^{Tag} : On receiving an identifier ID , a file block m_i along with filename $Fname$, $\mathcal{C}$ responds with a verifiable tag σ by running **TagGen**($params, ID, SK, m_i, i, Fname$).

Forge Phase. $\mathcal{A}_1$ outputs a tag σ' for a file block m'_i with block index i' and filename $Fname'$ under the identifier ID' and PK' . $\mathcal{A}_1$ wins if **TagVerify**($params, ID', PK', m'_i, i', Fname', \sigma'$) = 1 and its queries are compliance with the following restrictions:

- $\mathcal{A}_1$ has never queried $O^{PrivateKey}$ on ID' ;
- $\mathcal{A}_1$ has never queried $O^{PartialKey}$ on ID' and $O^{ReplacePublicKey}$ on $(ID', \cdot)$ at same time;
- $\mathcal{A}_1$ has never queried O^{Tag} on $(ID', m'_i, i', Fname')$.

The advantage of $\mathcal{A}_1$ winning the Tag-EUF-CMA Game-I is defined by $Adv_{\mathcal{A}_1}^{Tag-EUF-CMA} = Pr[\mathcal{A}_1 \text{ wins}]$.

Tag-EUF-CMA Game-II. This game is played by the challenger $\mathcal{C}$ and $\mathcal{A}_2$. It is similar to the Tag-EUF-CMA Game-I, but has some differences: (1) $\mathcal{C}$ gives both $params$ and msk to $\mathcal{A}_2$; (2) $\mathcal{A}_2$ is not provided with the oracles $O^{PartialKey}$ and $O^{ReplacePublicKey}$.

Forge Phase. $\mathcal{A}_2$ outputs a tag σ' of a file block m'_i with block index i' and filename $Fname$ under the identity ID' and PK' . $\mathcal{A}_2$ wins if **TagVerify**($params, ID', PK', m'_i, i', Fname', \sigma'$) = 1 and its queries are compliance with the following restrictions:

- $\mathcal{A}_2$ has never queried $O^{PrivateKey}$ on ID' ;
- $\mathcal{A}_2$ has never queried O^{Tag} on $(ID', m'_i, i', Fname')$.

The advantage of $\mathcal{A}_2$ winning the Tag-EUF-CMA Game-II is defined by $Adv_{\mathcal{A}_2}^{Tag-EUF-CMA} = Pr[\mathcal{A}_2 \text{ wins}]$.

Definition 2: For an LB-CL-DIA scheme, if the advantage of any adversary $\mathcal{A} \in \{\mathcal{A}_1, \mathcal{A}_2\}$ winning either the Tag-EUF-CMA Game-I or the Tag-EUF-CMA Game-II is negligible, the scheme achieves the Tag-EUF-CMA security.

Proof-EUF-CMA Security. This security is formalized through the following adversarial game:

Proof-EUF-CMA Game. This game is played by the challenger $\mathcal{C}$ and $\mathcal{A}_3$.

Setup Phase. $\mathcal{C}$ runs the **Setup** algorithm to create $params$ and msk , and gives $params$ to $\mathcal{A}_3$.

Query Phase. $\mathcal{A}_3$ is allowed to query the oracles $O^{CreateUser}$, $O^{PartialKey}$, $O^{PrivateKey}$, $O^{ReplacePublicKey}$ and O^{Tag} . $\mathcal{C}$ answers these queries in the same way as previously described.

Challenge Phase. $\mathcal{C}$ produces a random challenge $Chal$ with challenge identity ID' and filename $Fname'$ to $\mathcal{A}_3$, where $Chal$ includes at least one index i of a file block m_i^* such that $\mathcal{A}_3$ has never queried O^{Tag} on $(ID', m_i^*, i, Fname')$. The restriction on $Chal$ models the situation that the challenge contains some missing or damaged file blocks, which is not made in the existing security model. Our restriction is necessary, otherwise the adversary will know the verifiable tags of all challenge file blocks.

Forge Phase. $\mathcal{A}_3$ generates the auditing proof $Proof$ according to $Chal$. $\mathcal{A}_3$ wins if **ProofVerify**

($params, Chal, Proof, PK'$) = 1 and its queries are compliance with the following restrictions:

- $\mathcal{A}_3$ has never queried $O^{PrivateKey}$ on ID' .
- $\mathcal{A}_3$ has never queried $O^{PartialKey}$ on ID' and $O^{ReplacePublicKey}$ on $(ID', \cdot)$ at same time.

The advantage of $\mathcal{A}_3$ winning the Proof-EUF-CMA Game is defined by $Adv_{\mathcal{A}_3}^{Proof-EUF-CMA} = Pr[\mathcal{A}_3 \text{ wins}]$.

Definition 3: For an LB-CL-DIA scheme, if the advantage of the adversary $\mathcal{A}_3$ winning the Proof-EUF-CMA Game is negligible, then the scheme satisfies the Proof-EUF-CMA security.

Data Privacy Security. Following [44], [45], we define the data privacy security for LB-CL-DIA through the following game:

Data Privacy Game. This game is played between a challenger $\mathcal{S}$ and $\mathcal{A}_4$.

Setup Phase. $\mathcal{S}$ is given the public parameters $params$. It gives $params$ to $\mathcal{A}_4$.

Query Phase. $\mathcal{A}_4$ is allowed to query the oracles $O^{CreateUser}$, $O^{PartialKey}$ and $O^{PrivateKey}$. $\mathcal{S}$ answers these queries in the same way as before.

Challenge Phase. $\mathcal{A}_4$ interacts with $\mathcal{S}$ as follows:

- $\mathcal{A}_4$ submits two equal-length files F_0, F_1 , and a target identifier ID . $\mathcal{S}$ gets the private key of ID and executes the **TagGen** algorithm to generate verifiable tags $\sigma_0 = \{\sigma_{0i}\}$ and $\sigma_1 = \{\sigma_{1i}\}$ corresponding to F_0 and F_1 in turn and returns them to $\mathcal{A}_4$.
- $\mathcal{A}_4$ creates a challenge message $Chal$ including the filename $Fname$, and the index set of file blocks to be audited, and then sends $Chal$ to $\mathcal{S}$.
- $\mathcal{S}$ chooses a random bit $b \in \{0, 1\}$ and simulates the auditing proof $Proof^*$ according to F_b . Then, it returns $Proof^*$ to $\mathcal{A}_4$.

Guess Phase. $\mathcal{A}_4$ outputs a guess bit b' .

The advantage of $\mathcal{A}_4$ winning the data privacy game is defined by $Adv_{\mathcal{A}_4}^{DataPrivacy} = Pr[b = b' | - 1/2]$.

Definition 4: An LB-CL-DIA scheme achieves data privacy security if there exists a polynomial-time simulator $\mathcal{S}$ such that for any adversary $\mathcal{A}_4$, the following random variables are computationally indistinguishable:

- 1) Real audit interaction view:

$$View_{\mathcal{A}_4}(CS_{PR}, Chal, ID, F, Fname, \sigma, Proof^*)$$

where R denote the random coins the scheme uses, σ denotes the set of verifiable tags according to F .

- 2) Simulated view: $\mathcal{S}(Chal, ID, Fname)$.

This definition indicates that the simulator $\mathcal{S}$, using only public inputs ($Chal, ID, Fname$), generates an audit proof indistinguishable from the real audit interaction view. The adversary $\mathcal{A}_4$ has a negligible advantage in extracting valid information about the file blocks through the audit process.

IV. THE PROPOSED LB-CL-DIA SCHEME

A. Concrete Construction

The concrete construction of our LB-CL-DIA scheme is as follows:

- **Setup**(k): The KGC executes the subsequent steps:
 - 1) Generates an elliptic curve group $\mathbb{G}$ with k -bit prime order q and generator P ;
 - 2) Randomly chooses an $s \in \mathbb{Z}_q^*$ and computes $P_{pub} = sP$;
 - 3) Selects hash functions: $H_1 : \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q^*$, $H_2 : \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q^*$, and $H_3 : \{0, 1\}^* \times \{0, 1\}^* \times \mathbb{G} \times \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{Z}_q^*$;
 - 4) Sets $params = \{q, \mathbb{G}, P, P_{pub}, H_1, H_2, H_3\}$ and $msk = s$.

- **PartialKeyGen**($params, s, ID$): The KGC creates a partial secret key PSK for the DO as follows:

- 1) Chooses a random $r \in \mathbb{Z}_q^*$, calculates $R = rP$ and $d = r + s \cdot h_1$ where $h_1 = H_1(ID, R, P_{pub})$;
- 2) Sets $PSK = (d, R)$.

The DO can validate PSK by verifying the equation:

$$dP \stackrel{?}{=} R + H_1(ID, R, P_{pub})P_{pub}. \quad (1)$$

- **UserKeyGen**($params, PSK$): The DO generates his/her key pair (PK, SK) with $PSK = (d, R)$ as follows:

- 1) Chooses a random $x \in \mathbb{Z}_q^*$ and calculates $X = xP$;
- 2) Sets $PK = (X, R)$ and $SK = (x, d)$.

- **TagGen**($params, ID, SK, \{m_i, i\}_{i \in [1, n]}, Fname$): The DO and the EN perform the algorithm to create a verifiable tag σ_i for the file block $m_i \in \mathbb{Z}_q^*$ as follows:

- 1) The DO chooses $a, u_i, \dots, u_n \in \mathbb{Z}_q^*$ and computes $u'_i = a + u_i$. It transmits $\{u'_i\}_{i \in [1, n]}$ and a request for a partial tag to the EN.
- 2) The EN returns a partial tag $\hat{\sigma} = \{U'_i\}_{i \in [1, n]}$, where $U'_i = u'_i P$;
- 3) The DO computes $A = aP$, and $U_i = U'_i - A$. Then, it can get $\varsigma_i = xh_{2i} + (u_i + d)m_i h_{3i}$ by his/her secret key $SK = (x, d)$ where $h_{2i} = H_2(ID, Fname, i, PK)$ and $h_{3i} = H_3(ID, i, U_i, PK)$, and sets the verifiable tag of each block $\sigma_i = \{U_i, \varsigma_i\}$ and $\sigma = \{\sigma_i\}$, where $i \in [1, n]$.

- **TagVerify**($params, ID, PK, m_i, i, Fname, \sigma_i$): The CSP verifies the validation of a verifiable tag $\sigma_i = (U_i, \varsigma_i)$ by the following equation:

$$\varsigma_i P \stackrel{?}{=} h_{2i} X + h_{3i} m_i U_i + h_{3i} m_i W, \quad (2)$$

where $h_{2i} = H_2(ID, Fname, i, PK)$, $h_{3i} = H_3(ID, i, U_i, PK)$ and $W = R + H_1(ID, R, P_{pub})P_{pub}$.

- **Store**($params, ID, SK, F, Fname$): The DO executes the following steps to generate his storage data and then interacts with the CSP to agree on uploading data.

- 1) The DO splits the data file F into n blocks so that $F = \{m_1, m_2, \dots, m_i, \dots, m_n\}$, where $m_i \in \mathbb{Z}_q^*$ ($1 \leq i \leq n$);
- 2) The DO interacts with the EN to run **TagGen** to generate a verifiable tag $\sigma = \{\sigma_i\}$ for each m_i ($1 \leq i \leq n$) in F ;
- 3) The DO sets all tags of F as $\Phi = \{\sigma_i\}_{1 \leq i \leq n}$, and transmits the storage data $\Omega = (ID, Fname, F, n, \Phi)$ to the CSP.

After the receipt of Ω , the CSP runs **TagVerify** to verify the tag of each file block. If all verifications pass, the CSP accepts to store Ω and returns 1; Otherwise, it rejects to store Ω and returns 0.

Once successfully uploading data, the DO sends the metadata of the uploaded data (including his/her identifier ID , the filename $Fname$ and the quantity n of file blocks) to both the EN and the BN.

- **ChalGen**($params, ID, Fname, n$): To launch DIA, the EN initiates an auditing request (including the DO's identity ID and the filename $Fname$ of the file to be audited) to the BN. Then, the BN creates a challenge message $Chal$ as follows:

- 1) Creates $Q = \{(i, v_i)\}_{i \in I}$, where I is a random subset of $[1, n]$ and $v_i \in \mathbb{Z}_q^*$ ($i \in I$) are randomly-selected values;
- 2) Sets $Chal = (ID, Fname, Q)$.

- **ProofGen**($params, Chal$): Upon receiving a challenge $Chal = (ID, Fname, Q)$, the CSP computes the corresponding auditing proof $Proof$ as follows:

- 1) Locates all file blocks and the corresponding verifiable tags $\{m_i, \sigma_i = (U_i, \varsigma_i)\}_{i \in I}$ according to $Chal$;
- 2) Randomly chooses $t \in \mathbb{Z}_q^*$, computes $h_{3i} = H_3(ID, i, U_i, PK)$, $\zeta = t \sum_{i \in I} v_i m_i h_{3i}$, $\xi = t \sum_{i \in I} v_i m_i h_{3i} U_i$, $\delta = t \sum_{i \in I} v_i \varsigma_i$, $T = tX$;
- 3) Sets $Proof = (\zeta, \xi, \delta, T)$.

- **ProofVerify**($params, Chal, Proof, PK$): Given $Chal = (ID, Fname, Q)$, $Proof = (\zeta, \xi, \delta, T)$ and the DO's public key $PK = (X, R)$, the BN operates in the following manner:

- 1) Computes $W = R + H_1(ID, R, P_{pub})P_{pub}$ and $\varphi = \sum_{i \in I} v_i h_{2i}$;
- 2) Checks if the following equation holds:

$$\delta P \stackrel{?}{=} \varphi T + \xi + \zeta W. \quad (3)$$

- 3) Outputs 1 if Eqn. (3) is true, or 0 if it is false.

B. Scheme Workflow

The workflow is shown in Fig. 2. We provide a brief description of how our scheme is executed. Its workflow is separated into three phases. More technical details can be found in Section VI.B.

1) **Setup and Key Generation Phase**: The KGC performs the algorithm **Setup** to initialize the system and runs the algorithm **PartialKeyGen** to create a partial secret key PSK for the DO. Then, the DO executes the algorithm **UserkeyGen** to produce a public/secret key pair (PK, SK) .

2) **Data Store Phase**: The DO performs the algorithm **Store** to upload his/her data. If the returned result is 1, the DO sends the metadata (including the identifier ID , the filename $Fname$, and the quantity n of file blocks) to the EN and the BN respectively. Meanwhile, the DO submits $(ID, Fname, \theta = 1)$ to the smart contract **StoreRes** to generate a transaction TX, in which $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$ is a publicly available hash function.

Integrity Auditing Phase:

1) Upon receiving an auditing request $(ID, Fname)$ from the EN, the BN locates the metadata related to $(ID, Fname)$

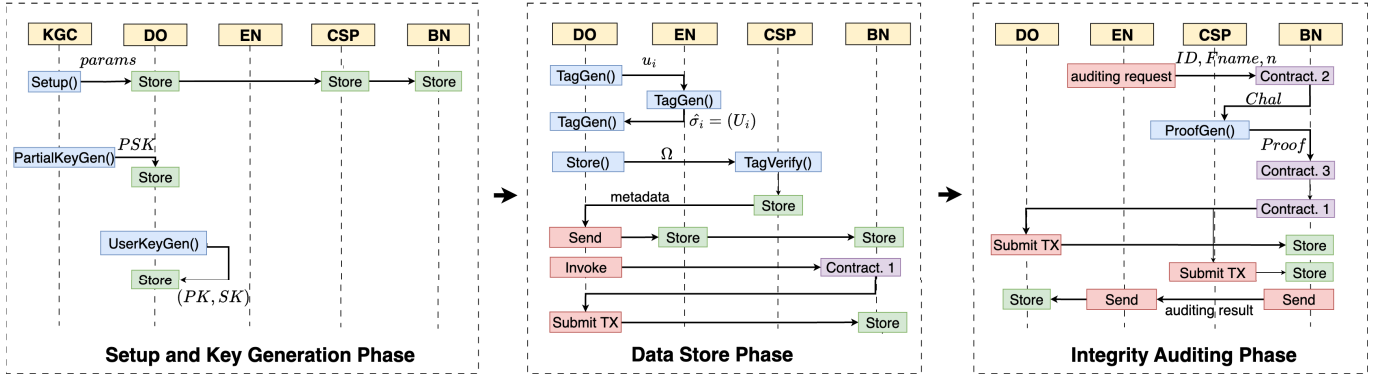


Fig. 2. Workflow of LB-CL-DIA.

Contract 1 StoreRes (Storage request)**Input:** $ID, Fname, \theta$ **Output:** TX

- 1 if $(\theta = 1)$ then
 - 2 Set $From = Accout_{DO}$;
 - 3 Set $To = Accout_{CSP}$;
 - 4 Set $Data = H(ID) || H(Fname) || \theta || Pay_{store}$;
- 5 else
 - 6 Set $From = Accout_{DO}$;
 - 7 Set $To = Accout_{CSP}$;
 - 8 Set $Data = H(ID) || H(Fname) || \theta || Pay_{store}$;
- 9 Set $TX = From || To || Data$;
- 10 return TX ;

Contract 2 ChalRes (Challenge request)**Input:** $params, ID, Fname, n$ **Output:** $Chal$

- 1 Randomly choose a subset I from $[n]$;
- 2 for $i \in I$ do
 - 3 For each number i , choose a $v_i \in \mathbb{Z}_q^*$;
 - 4 Set $Q = \{i, v_i\}_{i \in I}$;
- 5 Set $Chal = (ID, Fname, Q)$;
- 6 Clear the cache on the chain;
- 7 return $Chal$;

and then invokes the smart contract **ChalRes** with input $(params, ID, Fname, n)$ to create a challenge message $Chal$. By using keccak256, the smart contract **ChalRes** generates a pseudo-randomized $I \subset [n]$. According to Ateniese et al. [2], selecting 4.6% of file blocks can detect server misbehavior with over 99% probability. To enhance detection reliability and address potential factors such as reducing false negatives and improving system trust, we use 10% of the file blocks for the challenge. Then, the smart contract chooses a random number v_i from $\mathbb{Z}_q^*$ for each $i \in I$ and transmits $(ID, Fname, \{i, v_i\}_{i \in I})$ as a challenge message to the CSP.

2) Upon the receipt of $Chal$, the CSP runs the algorithm **ProofGen** $(params, Chal)$ to generate an auditing proof $Proof$ and then sends $(Proof, Fname, ID)$ to the BN for verification.

Contract 3 ProVerify (Proof verify)**Input:** $params, Proof, Fname, ID, Chal$ **Output:** θ

- 1 Compute $\theta = \mathbf{ProofVerify}(params, Chal, Proof, PK)$;
- 2 Invoke the smart contract **StoreRes** on input $(ID, Fname, \theta)$;
- 3 return θ ;

3) The verification of $Proof$ is done by the smart contract **ProVerify**. Let the auditing result be θ . the smart contract **ProVerify** invokes the smart contract **StoreRes** on input $(ID, Fname, \theta)$ to generate a transaction TX. If $\theta = 1$, the DO needs to pay the storage fee Pay_{store} to the CSP; Otherwise, the CSP needs to pay the compensation fee Pay_{comp} to the DO. Finally, the BN returns the auditing result to the EN, which in turn is sent to the DO.

V. CORRECTNESS AND SECURITY ANALYSIS

A. Correctness

For Eqn. (1), the correctness can be confirmed as:

$$\begin{aligned}
 dP &= (r + sh_1)P \\
 &= rP + sh_1P \\
 &= R + H_1(ID, R, P_{pub})P_{pub}
 \end{aligned} \tag{4}$$

For Eqn. (2), the correctness can be confirmed as:

$$\begin{aligned}
 \varsigma_i P &= (xh_{2i} + (u_i + d)m_i h_{3i})P \\
 &= xh_{2i}P + u_i m_i h_{3i}P + (r + sh_1)m_i h_{3i}P \\
 &= h_{2i}X + m_i h_{3i}U_i + m_i h_{3i}(R + h_1 P_{pub}) \\
 &= h_{2i}X + h_{3i}m_i U_i + h_{3i}m_i W
 \end{aligned} \tag{5}$$

For Eqn. (3), the correctness can be confirmed as:

$$\begin{aligned}
 \delta P &= t \sum_{i \in I} v_i \varsigma_i P \\
 &= t \sum_{i \in I} v_i (xh_{2i} + (u_i + d)m_i h_{3i})P \\
 &= t \sum_{i \in I} v_i xh_{2i}P + t \sum_{i \in I} v_i u_i m_i h_{3i}P \\
 &\quad + t \sum_{i \in I} v_i (r + sh_1)m_i h_{3i}P
 \end{aligned}$$

$$\begin{aligned}
&= t \sum_{i \in I} v_i h_{2i} X + t h_{3i} \sum_{i \in I} v_i m_i U_i \\
&\quad + t \sum_{i \in I} v_i m_i h_{3i} (R + h_1 P_{pub}) \\
&= \varphi T + \xi + \zeta W
\end{aligned} \tag{6}$$

B. Security Proofs

Theorem 1: Based on the ECDLP assumption, our LB-CL-DIA scheme achieves the Tag-EUF-CMA security in the random oracle model.

Theorem 1 is followed by Lemma 1 and Lemma 2.

Lemma 1: If an adversary $\mathcal{A}_1$ wins the Tag-EUF-CMA Game-I with advantage ε , then an algorithm $\mathcal{B}$ can be built to solve the ECDLP with probability $\left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H1} \cdot q_3} \cdot \varepsilon$, where q_1 , q_2 , q_3 and q_{H1} denote the number of $O^{PartialKey}$, $O^{PrivateKey}$, $O^{CreateUser}$ and O^{H1} queries, respectively.

Proof: For an instance $(q, \mathbb{G}, P, P^* = sP)$ of the ECDLP, $\mathcal{B}$'s goal is to find s .

Setup Phase. $\mathcal{B}$ sends $params = \{q, \mathbb{G}, P, P_{pub}, H_1, H_2, H_3\}$ to $\mathcal{A}_1$, where $P_{pub} = P^*$, and H_1, H_2, H_3 are three hash functions simulated by $\mathcal{B}$ as the random oracles O^{H1} , O^{H2} , O^{H3} . Besides, $\mathcal{B}$ selects a random $J \in [1, q_3]$.

Query Phase. $\mathcal{A}_1$ adaptively makes various queries. $\mathcal{B}$ maintains some lists $L_{H1} = \{(ID_j, R_j, P_{pub}, h_{1j})\}$, $L_{H2} = \{(ID_j, Fname, PK_j, h_{2j})\}$, $L_{H3} = \{(ID_j, i, U_i, PK_j, h_{3ij})\}$ and $L_{Cuser} = \{(ID_j, X_j, R_j, x_j, d_j)\}$.

Queries to O^{H1} . When $\mathcal{A}_1$ asks a query $H_1(ID_j, R_j, P_{pub})$, $\mathcal{B}$ returns h_{1j} if a tuple $(ID_j, R_j, P_{pub}, h_{1j})$ exists in L_{H1} . Otherwise, $\mathcal{B}$ chooses $h_{1j} \in \mathbb{Z}_q^*$, adds $(ID_j, R_j, P_{pub}, h_{1j})$ to L_{H1} and returns h_{1j} to $\mathcal{A}_1$.

Queries to O^{H2} . When $\mathcal{A}_1$ asks a query $H_2(ID_j, Fname, i, PK_j)$, $\mathcal{B}$ returns h_{2ij} if a tuple $(ID_j, Fname, i, PK_j, h_{2ij})$ exists in L_{H2} . Otherwise, $\mathcal{B}$ chooses $h_{2ij} \in \mathbb{Z}_q^*$, adds $(ID_j, Fname, i, PK_j, h_{2ij})$ to L_{H2} and returns h_{2ij} to $\mathcal{A}_1$.

Queries to O^{H3} . When $\mathcal{A}_1$ asks a query $H_3(ID_j, i, U_i, PK_j)$, $\mathcal{B}$ returns h_{3ij} if a tuple $(ID_j, i, U_i, PK_j, h_{3ij})$ exists in L_{H3} . If not, $\mathcal{B}$ selects $h_{3ij} \in \mathbb{Z}_q^*$, adds $(ID_j, i, U_i, PK_j, h_{3ij})$ to L_{H3} and returns h_{3ij} to $\mathcal{A}_1$.

Queries to $O^{CreateUser}$. When $\mathcal{A}_1$ asks a query on ID_j , $\mathcal{B}$ returns $PK_j = (X_j, R_j)$ if $(ID_j, X_j, R_j, x_j, d_j)$ exists in L_{Cuser} . Otherwise, $\mathcal{B}$ does the following steps:

- If $ID_j = ID_J$, $\mathcal{B}$ randomly selects $r_j, h_{1j}, x_j \in \mathbb{Z}_q^*$ and defines $R_j = r_j P$, $H_1(ID_j, R_j, P_{pub}) = h_{1j}$ and $X_j = x_j P$. Then, $\mathcal{B}$ adds $(ID_j, R_j, P_{pub}, h_{1j})$ and $(ID_j, X_j, R_j, x_j, \perp)$ to L_{H1} and L_{Cuser} respectively. Finally, $\mathcal{B}$ returns $PK_j = (X_j, R_j)$ to $\mathcal{A}_1$.
- Else, $\mathcal{B}$ randomly selects $x_j, h_{1j}, d_j \in \mathbb{Z}_q^*$ and computes $X_j = x_j P$, $R_j = d_j P - h_{1j} P_{pub}$. Then, $\mathcal{B}$ adds $(ID_j, X_j, R_j, x_j, d_j)$ and $(ID_j, R_j, P_{pub}, h_{1j})$ to L_{Cuser} and L_{H1} respectively, and returns $PK_j = (X_j, R_j)$.

Queries to $O^{PartialKey}$. When $\mathcal{A}_1$ asks a query to this oracle on ID_j , if $ID_j = ID_J$, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns d_j .

Queries to $O^{ReplacePublicKey}$. When $\mathcal{A}_1$ asks a query on ID_j and $PK'_j = (X'_j, R'_j)$, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and updates it by $(ID_j, X'_j, R'_j, \perp, \perp)$.

Queries to $O^{PrivateKey}$. When $\mathcal{A}_1$ asks a query to this oracle on ID_j , if $ID_j = ID_J$, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns $SK_j = (x_j, d_j)$.

Queries to O^{Tag} . When $\mathcal{A}_1$ asks a query to this oracle on $(ID_j, m_i, i, Fname)$, $\mathcal{B}$ performs as follows:

- If $ID_j \neq ID_J$, $\mathcal{B}$ chooses $u_i \in \mathbb{Z}_q^*$ and computes $U_i = u_i P$. Then, it retrieves $(ID_j, R_j, P_{pub}, h_{1j})$, $(ID_j, Fname, i, PK_j, h_{2ij})$, $(ID_j, i, U_i, PK_j, h_{3ij})$ and $(ID_j, X_j, R_j, x_j, d_j)$ from L_{H1} , L_{H2} , L_{H3} and L_{Cuser} , respectively. Finally, it computes $\varsigma_i = x_j h_{2ij} + (u_i + d_j) m_i h_{3ij}$ and returns $\sigma_i = (U_i, \varsigma_i)$.
- If $ID_j = ID_J$, $\mathcal{B}$ recovers $(ID_j, R_j, P_{pub}, h_{1j})$ from L_{H1} and $(ID_j, X_j, R_j, x_j, d_j)$ from L_{Cuser} , respectively. Then, it selects $\varsigma_i, h_{2ij}, h_{3ij} \in \mathbb{Z}_q^*$ and sets $U_i = (m_i h_{3ij})^{-1} (\varsigma_i P - h_{2ij} X_j - m_i h_{3ij} (R_j + h_{1j} P_{pub}))$. Finally, it returns $\sigma_i = (U_i, \varsigma_i)$, adds $(ID_j, Fname, i, PK_j, h_{2ij})$ to L_{H2} and $(ID_j, i, U_i, PK_j, h_{3ij})$ to L_{H3} , respectively.

Forgery. $\mathcal{A}_1$ returns a forgery $(ID_j^*, m_i^*, i^*, \sigma_i^* = (U_i^*, \varsigma_i^*))$. If $ID_j^* \neq ID_J$, $\mathcal{B}$ aborts the simulation. Otherwise, the forgery should satisfy Eqn. (2). Thus, we have

$$\varsigma_i^* P = h_{2ij}^* X_j^* + h_{3ij}^* m_i^* U_i^* + h_{3ij}^* m_i^* (R_j^* + h_{1j}^* P_{pub}). \tag{7}$$

According to the forking lemma [46], and by repeating the O^{H1} query, $\mathcal{A}_1$ can also generate another valid forged tag (U_i^*, ς_i^*) in the same way. So, we also have

$$\varsigma_i'^* P = h_{2ij}^* X_j^* + h_{3ij}^* m_i^* U_i'^* + h_{3ij}^* m_i^* (R_j^* + h_{1j}^* P_{pub}). \tag{8}$$

By combining Eqn. (7) and Eqn. (8), we can derive

$$\varsigma_i'^* - \varsigma_i^* = s(h_{1j}^* - h_{1j}^*) m_i^* h_{3ij}^* + (u_i' - u_i^*) m_i^* h_{3ij}^*.$$

Therefore, $s = \frac{\varsigma_i'^* - \varsigma_i^*}{(h_{1j}' - h_{1j}^* + u_i' - u_i^*) m_i^* h_{3ij}^*}$ is a correct solution to the ECDLP.

Probability analysis. To assess the success probability that $\mathcal{B}$ solves the given ECDLP, we define three events:

- E_1 : $\mathcal{A}_1$ does not query $O^{PrivateKey}$ or $O^{PartialKey}$ on ID_J during the Query Phase.
- E_2 : $\mathcal{A}_1$ makes a forgery $(ID_j^*, \cdot, \cdot, \cdot)$ such that $ID_j^* = ID_J$.
- E_3 : $\mathcal{A}_1$ successfully outputs two valid tag.

From the above simulation, it is easy to see that the probability of E_1 and E_2 are $P(E_1) = \left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2}$ and $P(E_2) = \frac{1}{q_3}$, respectively.

We already know that the advantage of $\mathcal{A}_1$ outputting a valid forgery is ε . By the forking lemma [46], the probability that $\mathcal{A}_1$ outputs two valid forgery is $P(E_3) = \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H1}} \cdot \varepsilon$.

Therefore, the probability that $\mathcal{B}$ addresses the ECDLP is bounded by $\varepsilon' \geq \left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2} \cdot \frac{1}{q_3} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H1}} \cdot \varepsilon$. $\square$

Lemma 2: If an adversary $\mathcal{A}_2$ wins the Tag-EUF-CMA Game-II with advantage ε , then an algorithm $\mathcal{B}$ can be built to solve the ECDLP with probability $\left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H2} \cdot q_3} \cdot \varepsilon$, where q_1 , q_2 , q_3 and q_{H2} denote the number of $O^{PartialKey}$, $O^{PrivateKey}$, $O^{CreateUser}$ and O^{H2} queries, respectively.

Proof: For an instance $(q, \mathbb{G}, P, P^* = aP)$ of the ECDLP, $\mathcal{B}$'s goal is to find a .

Setup Phase. $\mathcal{B}$ randomly selects $s \in \mathbb{Z}_q^*$, sends $msk = s$ and $params = \{q, \mathbb{G}, P, P_{pub}, H_1, H_2, H_3\}$ to $\mathcal{A}_2$, where $P_{pub} = sP$.

and H_1, H_2, H_3 are three hash functions simulated by $\mathcal{B}$ as the random oracles $O^{H_1}, O^{H_2}, O^{H_3}$. Besides, $\mathcal{B}$ selects a random $J \in [1, q_2]$.

Query Phase. $\mathcal{A}_2$ adaptively makes various queries. $\mathcal{B}$ maintains some lists L_{H1}, L_{H2}, L_{H3} and L_{Cuser} .

Queries to $O^{H_1}, O^{H_2}, O^{H_3}, O^{PrivateKey}$ and O^{Tag} follow the same pattern as described previously.

Queries to $O^{CreateUser}$. When $\mathcal{A}_2$ asks a query on ID_j , $\mathcal{B}$ returns $PK_j = (X_j, R_j)$ if $(ID_j, X_j, R_j, x_j, d_j)$ exists in L_{Cuser} . Alternatively, $\mathcal{B}$ executes the subsequent procedures:

- If $ID_j = ID_J$, $\mathcal{B}$ randomly selects $r_j, h_{1j} \in \mathbb{Z}_q^*$ and defines $R_j = r_j P$, $X_j = P^*$, $d_j = r_j + sh_{1j}$ and $H_1(ID_j, R_j, P_{pub}) = h_{1j}$. Then, $\mathcal{B}$ adds $(ID_j, R_j, P_{pub}, h_{1j})$ and $(ID_j, X_j, R_j, \perp, d_j)$ to L_{H1} and L_{Cuser} respectively, and returns $PK_j = (X_j, R_j)$.
- If $ID_j \neq ID_J$, $\mathcal{B}$ randomly selects $x_j, h_{1j}, r_j \in \mathbb{Z}_q^*$ and computes $X_j = x_j P$, $R_j = r_j P$, and $d_j = r_j + sh_{1j}$. Then, $\mathcal{B}$ adds $(ID_j, X_j, R_j, x_j, d_j)$ and $(ID_j, R_j, P_{pub}, h_{1j})$ to L_{Cuser} and L_{H1} respectively, and returns $PK_j = (X_j, R_j)$.

Forgery. $\mathcal{A}_2$ returns a forgery $(ID_j^*, m_i^*, i^*, \sigma_i^* = (U_i^*, \varsigma_i^*))$. If $ID_j^* \neq ID_J$, $\mathcal{B}$ aborts the simulation. Otherwise, the forgery should satisfy Eqn. (2). Thus, we have

$$\varsigma_i^* P = h_{2ij}^* X_j^* + h_{3ij}^* m_i^* U_i^* + h_{3ij}^* m_i^* (R_j^* + h_{1j}^* P_{pub}). \quad (9)$$

According to the forking lemma [46], and by repeating the O^{H_2} query, $\mathcal{A}_2$ can also generate another valid forged tag $(U_i^*, \varsigma_i^{*'})$ in the same way. So, we also have

$$\varsigma_i^{*'} P = h_{2ij}^* X_j^* + h_{3ij}^* m_i^* U_i^{*'} + h_{3ij}^* m_i^* (R_j^* + h_{1j}^* P_{pub}). \quad (10)$$

By combining Eqn. (9) and Eqn. (10), we can derive

$$\varsigma_i^{*'} - \varsigma_i^* = a(h_{2ij}^{*'} - h_{2ij}^*) + (u_i^{*'} - u_i^*) h_{3ij}^* m_i^*.$$

Therefore, $a = \frac{\varsigma_i^{*'} - \varsigma_i^* + (u_i^{*'} - u_i^*) h_{3ij}^* m_i^*}{(h_{2ij}^{*'} - h_{2ij}^*)}$ is a correct solution to the ECDLP.

Probability analysis. Now, we assess the probability that $\mathcal{B}$ solves the given ECDLP by utilizing $\mathcal{A}_2$. To do so, we define the following three events that contribute to the success of $\mathcal{B}$.

- E_1 : $\mathcal{A}_2$ does not query $O^{PrivateKey}$ on ID_J during the Query Phase.
- E_2 : $\mathcal{A}_2$ makes a forgery $(ID_j^*, \cdot, \cdot, \cdot)$ such that $ID_j^* = ID_J$.
- E_3 : $\mathcal{A}_2$ successfully outputs two valid tag.

From the above simulation, it is easy to see that the probability of E_1 and E_2 are $P(E_1) = \left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2}$ and $P(E_2) = \frac{1}{q_3}$, respectively.

We already know that the advantage of $\mathcal{A}_2$ outputting a valid forgery is ε . By the forking lemma [46], the probability that $\mathcal{A}_2$ outputs two valid forgery is $P(E_3) = \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H2}} \cdot \varepsilon$.

Therefore, $\mathcal{B}$ can solve the ECDLP with a probability $\varepsilon' \geq \left(1 - \frac{1}{q_1 + q_2 + 1}\right)^{q_1 + q_2} \cdot \frac{1}{q_3} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H2}} \cdot \varepsilon$. $\square$

Theorem 2: Based on the ECDLP assumption, our LB-CL-DIA scheme achieves the Proof-EUF-CMA security in the random oracle model. Specifically, if an adversary $\mathcal{A}_3$ wins the Proof-EUF-CMA Game with advantage ε , then an algorithm $\mathcal{B}$ can be built to solve the ECDLP with probability $\left(1 - \frac{1}{q_1 + q_2 + q_4 + 1}\right)^{q_1 + q_2 + q_4} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H1}} \cdot \varepsilon$, where q_1, q_2, q_4 and

q_{H1} denote the number of $O^{PartialKey}$, $O^{PrivateKey}$, O^{Tag} and O^{H_1} queries, respectively.

Proof: For an instance $(q, \mathbb{G}, P, P^* = sP)$ of the ECDLP, $\mathcal{B}$'s goal is to find s .

Setup Phase. $\mathcal{B}$ sends $params = \{q, \mathbb{G}, P, P_{pub}, H_1, H_2, H_3\}$ to $\mathcal{A}_3$, where $P_{pub} = P^*$ (which implies $msk = s$) and H_1, H_2, H_3 are three hash functions simulated by $\mathcal{B}$ as the random oracles $O^{H_1}, O^{H_2}, O^{H_3}$. Besides, $\mathcal{B}$ selects two randoms $J \in [1, q_3]$ and $I \in [1, q_4]$.

Query Phase. $\mathcal{A}_3$ adaptively makes various queries. $\mathcal{B}$ maintains some lists L_{H1}, L_{H2}, L_{H3} and L_{Cuser} .

Queries to O^{H_1}, O^{H_2} , and O^{H_3} follow the same pattern as described previously.

Queries to $O^{CreateUser}$: When $\mathcal{A}_3$ asks a query on ID_j , $\mathcal{B}$ returns $PK_j = (X_j, R_j)$ if $(ID_j, X_j, R_j, x_j, d_j)$ exists in L_{Cuser} . Alternatively, $\mathcal{B}$ executes the subsequent procedures:

- If $ID_j = ID_J$, $\mathcal{B}$ randomly selects $r_j, h_{1j}, x_j \in \mathbb{Z}_q^*$ and defines $R_j = r_j P$, $H_1(ID_j, R_j, P_{pub}) = h_{1j}$ and $X_j = x_j P$. Then, $\mathcal{B}$ adds $(ID_j, R_j, P_{pub}, h_{1j})$ and $(ID_j, X_j, R_j, x_j, \perp)$ to L_{H1} and L_{Cuser} respectively, and returns $PK_j = (X_j, R_j)$.
- Else, $\mathcal{B}$ randomly selects $x_j, h_{1j}, d_j \in \mathbb{Z}_q^*$ and computes $X_j = x_j P$, $R_j = d_j P - h_{1j} P_{pub}$. Then, $\mathcal{B}$ adds $(ID_j, X_j, R_j, x_j, d_j)$ and $(ID_j, R_j, P_{pub}, h_{1j})$ to L_{Cuser} and L_{H1} respectively, and returns $PK_j = (X_j, R_j)$.

Queries to $O^{PartialKey}$. When $\mathcal{A}_3$ asks a query to this oracle on ID_j , if $ID_j = ID_J$, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns d_j .

Queries to $O^{ReplacePublicKey}$. When $\mathcal{A}_3$ asks a query on ID_j and $PK_j' = (X_j', R_j')$, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and updates it by $(ID_j, X_j', R_j', \perp, \perp)$.

Queries to $O^{PrivateKey}$. When $\mathcal{A}_3$ asks a query on ID_j , if $ID_j = ID_J$, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns $S K_j = (x_j, d_j)$.

Queries to O^{Tag} . When $\mathcal{A}_3$ asks a query on this oracle with $(ID_j, m_i, i, Fname)$, if $ID_j = ID_J$ and $m_i = m_I$, $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ does as follows:

- If $ID_j \neq ID_J$, $\mathcal{B}$ chooses $u_i \in \mathbb{Z}_q^*$ and computes $U_i = u_i P$. Then, $\mathcal{B}$ recovers the $(ID_j, R_j, P_{pub}, h_{1j})$, $(ID_j, Fname, i, PK_j, h_{2ij})$, $(ID_j, i, U_i, PK_j, h_{3ij})$ and $(ID_j, X_j, R_j, x_j, d_j)$ from the lists L_{H1}, L_{H2}, L_{H3} and L_{Cuser} respectively. Finally, $\mathcal{B}$ computes $\varsigma_i = x_j h_{2ij} + (u_i + d_j) m_i h_{3ij}$ and returns $\sigma_i = (U_i, \varsigma_i)$.
- If $ID_j = ID_J$, $\mathcal{B}$ recovers the $(ID_j, R_j, P_{pub}, h_{1j})$ from L_{H1} , and $(ID_j, X_j, R_j, x_j, d_j)$ from L_{Cuser} . Then, $\mathcal{B}$ selects $\varsigma_i, h_{2ij}, h_{3ij} \in \mathbb{Z}_q^*$ and sets $U_i = (m_i h_{3ij})^{-1} (\varsigma_i P - h_{2ij} X_j - m_i h_{3ij} (R_j + h_{1j} P_{pub}))$. Finally, $\mathcal{B}$ returns $\sigma_i = (U_i, \varsigma_i)$ to $\mathcal{A}_3$, adds $(ID_j, Fname, i, PK_j, h_{2ij})$ to L_{H2} and $(ID_j, i, U_i, PK_j, h_{3ij})$ to L_{H3} respectively.

Challenge. $\mathcal{B}$ generates an auditing challenge message $Chal = (ID_J, Fname, Q)$, where Q contains the index of the challenge block I .

Forgery. Upon receiving the $Chal$, $\mathcal{A}_3$ should generate a valid auditing proof $Proof = (\zeta, \xi, \varphi, \delta)$ which satisfies Eqn. (3). Thus, we have

$$\delta P = \varphi T + \xi + \zeta W, \quad (11)$$

where $T = tX_J$, $\varphi = t' \sum_{i \in I} v_i h_{2ij}$, $\xi = t' \sum_{i \in I} v_i m_i h_{3ij} U_i$, $\zeta = t' \sum_{i \in I} v_i m_i h_{3ij}$, $\delta = t' \sum_{i \in I} v_i \zeta_i$ and $W = R + H_1(ID_J, R_J, P_{pub})P_{pub}$.

According to the forking lemma [46], and by repeating the O^{H_1} query, $\mathcal{A}_3$ can also generate another $Proof' = (\zeta', \xi', \delta', T')$ in the same way. So, we also have

$$\delta' P = \varphi' T + \xi' + \zeta' W', \quad (12)$$

where $T' = t' X_J$, $\varphi' = \sum_{i \in I} v_i h_{2ij} = \varphi$, $\xi' = t' \sum_{i \in I} v_i m_i h_{3ij} U_i$, $\zeta' = t' \sum_{i \in I} v_i m_i h_{3ij}$, $\delta' = t' \sum_{i \in I} v_i \zeta_i$ and $W' = R + H_1(ID_J, R_J, P_{pub})P_{pub}$.

By combining Eqn. (11) and Eqn. (12), we can derive

$$(\delta' - \delta)P = \varphi T - \varphi T' + \xi' - \xi + \zeta' W' - \zeta W$$

so that

$$\begin{aligned} (\delta' - \delta) &= (t' - t)x\varphi \\ &+ (t' - t) \sum u_i v_i m_i h_{3ij} + s(h_1' \zeta' - h_1 \zeta). \end{aligned}$$

Therefore, $s = \frac{\delta' - \delta + (t' - t)x\varphi + (t' - t) \sum u_i v_i m_i h_{3ij}}{(h_1' - h_1)h_2 \zeta}$ is a correct solution to the ECDLP.

Probability analysis. Now, we assess the probability that $\mathcal{B}$ can solve the given ECDLP by utilizing $\mathcal{A}_3$. To do so, we define two events that contribute to the success of $\mathcal{B}$.

- E_1 : $\mathcal{A}_3$ does not query $O^{PrivateKey}$ or $O^{PartialKey}$ on ID_J during the Query Phase.
- E_2 : $\mathcal{A}_3$ successfully outputs two valid proofs.

From the above simulation, it is easy to see that the probability of E_1 is $P(E_1) = \left(1 - \frac{1}{q_1 + q_2 + q_4 + 1}\right)^{q_1 + q_2 + q_4}$.

We already know that the advantage of $\mathcal{A}_3$ outputting a valid proof is ε . By the forking lemma [46], the probability that $\mathcal{A}_3$ outputs two valid proofs is $P(E_2) = \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H_1}} \cdot \varepsilon$.

Therefore, $\mathcal{B}$ can solve the ECDLP with a probability $\varepsilon' \geq \left(1 - \frac{1}{q_1 + q_2 + q_4 + 1}\right)^{q_1 + q_2 + q_4} \cdot \left(1 - \frac{1}{e}\right) \cdot \frac{1}{q_{H_1}} \cdot \varepsilon$. $\square$

Theorem 3: Our LB-CL-DIA scheme preserves the data privacy security in the random oracle model.

Proof: We construct a poly-nomialtime simulator $\mathcal{S}$ to play the role of the challenger. Given public parameters $params$, $\mathcal{S}$ interacts with $\mathcal{A}_4$ in the following way:

Setup Phase. $\mathcal{S}$ forwards $params$ to $\mathcal{A}_4$.

Query Phase. $\mathcal{A}_4$ adaptively makes a series of queries to $O^{CreateUser}$, $O^{PartialKey}$, $O^{PrivateKey}$, O^{H_1} , O^{H_2} and O^{H_3} . As same as above, $\mathcal{S}$ maintains some initially-empty lists L_{H_1} , L_{H_2} , L_{H_3} and L_{Cuser} .

Queries to O^{H_1} , O^{H_2} , O^{H_3} follow the same pattern as described previously.

Queries to $O^{CreateUser}$. When $\mathcal{A}_4$ asks a query on ID_j , $\mathcal{S}$ returns $PK_j = (X_j, R_j)$ if $(ID_j, X_j, R_j, x_j, d_j)$ exists in L_{Cuser} . Otherwise, $\mathcal{S}$ randomly selects $d_j, x_j \in \mathbb{Z}_q^*$, defines $H_1(ID_j, R_j, P_{pub}) = h_{1j}$ and $R_j = d_j P - h_{1j} P_{pub}$, adds $(ID_j, X_j, R_j, x_j, d_j)$ to L_{Cuser} and returns $PK_j = (X_j, R_j)$.

Queries to $O^{PartialKey}$. When $\mathcal{A}_4$ asks a query on ID_j , $\mathcal{S}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns d_j .

Queries to $O^{PrivateKey}$. When $\mathcal{A}_4$ asks a query on ID_j , $\mathcal{S}$ finds $(ID_j, X_j, R_j, x_j, d_j)$ in L_{Cuser} and returns $SK_j = (x_j, d_j)$.

Challenge Phase. $\mathcal{A}_4$ interacts with $\mathcal{S}$ as follows:

- $\mathcal{A}_4$ selects two equal-length files F_0, F_1 , and a target identifier ID . If the ID does not appear in $O^{CreateUser}$, $\mathcal{S}$ needs to generate its public-privacy key pair. Then, $\mathcal{S}$ executes the **TagGen** algorithm to generate verifiable tag $\sigma_0 = \{\zeta_i, U_i\}_{i \in [1, n]}$ and $\sigma_1 = \{\zeta_i, U_i\}_{i \in [1, n]}$ corresponding to F_0 and F_1 in turn and returns them to $\mathcal{A}_4$.
- $\mathcal{A}_4$ randomly selects the index set of file blocks to be audited $(i, v_i)_{i \in I}$ and sends the challenge message $Chal$ to $\mathcal{S}$.
- $\mathcal{S}$ chooses a random bit $b \in \{0, 1\}$ and simulates the auditing proof of F_b . $\mathcal{S}$ randomly chooses $t^* \in \mathbb{Z}_q^*$ and computes $\zeta^* = t^* \sum_{i \in I} v_i m_{bi} h_{3, i}$, $\xi^* = t^* \sum_{i \in I} v_i m_{bi} h_{3, i} U_i$, $\delta^* = t^* \sum_{i \in I} v_i \zeta_i$, and $T^* = t^* X$. $\mathcal{S}$ returns the $Proof^* = (\zeta^*, \xi^*, \delta^*, T^*)$ of F_b .

Guess Phase. $\mathcal{A}_4$ outputs a guess bit b' .

We argue that the probability of $b = b'$ must be $\frac{1}{2}$, i.e., $\mathcal{A}_4$'s advantage is 0, due to the randomness of verifiable labels ζ^* , ξ^* , δ^* , and T^* . For each response, the above simulation explicitly sets $\zeta^* = t^* \sum_{i \in I} v_i m_{bi}$. Since t^* is a random value, ζ^* is indistinguishable from the actual labels, meaning that the simulation is perfect. Furthermore, if $\mathcal{A}_4$ launches multiple attacks on the same message, it can obtain the following linear combination of $\{m_1, \dots, m_I\}$:

$$\begin{cases} t_1^* v_{1,1} m_1 + \dots + t_1^* v_{1,I} m_I = \zeta_1^* \\ t_2^* v_{2,1} m_1 + \dots + t_2^* v_{2,I} m_I = \zeta_2^* \\ \vdots \\ t_I^* v_{I,1} m_1 + \dots + t_I^* v_{I,I} m_I = \zeta_I^*. \end{cases}$$

Due to the random choice of t_i^* , the linear combinations become underdetermined from the adversary's perspective, making it computationally infeasible for $\mathcal{A}_4$ to recover any meaningful information about the individual m_i . The same is true for the remaining three labels. Therefore, the simulation is perfectly indistinguishable from the real execution to the adversary $\mathcal{A}_4$, ensuring that it cannot tell whether it is interacting with the simulator or with the real system. In conclusion, the data privacy of our LB-CL-DIA scheme is guaranteed.

Theorem 4: Our solutions are resistant to EN and CSP collusion attacks.

Proof: When ENs collude with the CSP, the CSP can obtain information about the file F , including the stored data $\Omega = (ID, Fname, F, n, \Phi)$ and the random parameter u'_i from the EN. However, even though the EN participates in part of the tagging computation, it only operates on blinded inputs and does not have access to the original secret values u_i . This prevents the EN or CSP from directly accessing the core secret information. Under the ECDLP hardness assumption, the CSP cannot invert the secret values u_i and a from the public parameter U'_i , and thus cannot derive the user's private key (x, d) using the equation $\zeta_i = x h_{3i} + (u_i + d) m_i h_{2i}$. As a result, even if the EN is compromised, it cannot disclose the critical information needed to forge a complete valid tag. Since neither the CSP nor the compromised EN has access to the necessary original secrets, according to the **Proof-EUF-CMA** security model, they cannot collude to forge valid tags ζ_i or audit proofs that would pass verification. Additionally, audit challenges are randomly selected by the smart contract,

TABLE III
SYMBOLS IN COMPARISONS

Symbol	Definition
T_m	Time of a multiplication in $\mathbb{G}_1$
T_e	Time of an exponentiation in $\mathbb{G}_1$
T_c	Time of a multiplication in $\mathbb{G}_T$
T_p	Time of a bilinear pairing
T_r	Time to get a random number in $\mathbb{Z}_q^*$
T_{kh}	Time to get a random number in the blockchain
T_{pa}	Time of a point addition in $\mathbb{G}$
T_{pm}	Time of a scalar multiplication in $\mathbb{G}$
T_{mtp}	Time of a Map-To-Point function
T_h	Time of a hash function
$ \mathbb{G}_1 $	Length of an element in $\mathbb{G}_1$
$ \mathbb{G} $	Length of an element in $\mathbb{G}$
$ \mathbb{Z}_q^* $	Length of an element in $\mathbb{Z}_q^*$

while the EN can only initiate audit requests, ensuring that any tampered data block will inevitably be detected.

VI. EFFICIENCY ANALYSIS

A. Scheme Experiments and Comparisons

We assess the efficiency of our LB-CL-DIA scheme through a comparative analysis with some existing CL-DIA schemes [17], [18], [21], [22], [26], [27]. Because the CL-DIA scheme presented in [26] does not satisfy the Proof-EUF-CMA security, we do not include it in the comparison.

All compared schemes are experimented with using the Charm-Crypto cryptographic library [47]. The experimental platform is MacOS with an Apple M3 chip and 16GB of operating memory. For the schemes that rely on the bilinear pairing [17], [18], [21], [22], [26], [27], the experiments are simulated using the Type A bilinear pairing groups, where the size of one element in $\mathbb{Z}_q^*$ is 256 bits, and the size of one element in the bilinear pairing groups $\mathbb{G}_1$ and $\mathbb{G}_T$ is 512 bits and 1024 bits, respectively. For our scheme, the experiments are performed using the Secp256 elliptic curve group, where the length of one element in the elliptic curve additive group $\mathbb{G}$ is 320 bits. In addition, SHA-256 is chosen for the hash function implementation.

Table III lists the symbols used in the comparisons.

1) *Computation Cost Comparison*: This subsection provides a comparison between our proposed scheme and the CL-DIA schemes [17], [18], [21], [22], [26], [27] in terms of computation cost. For the purpose of facilitating comparison, the number of file blocks is assumed to be n .

Table IV gives the computation costs of the algorithms **TagGen**, **ChalGen**, **ProofGen** and **ProofVerify** in five schemes. Among them, [17], [18], [21], [22] are pairing-based data integrity auditing schemes under the CLC setting, while [26] is a pairing-free scheme within the same CLC framework, and [27] is a blockchain-based data integrity auditing scheme. It is worth noting that when listing the computational cost of each algorithm, the computational overhead of integer operations is omitted in this paper. This is because their computational cost is negligible compared to operations such as addition/multiplication of grouped elements, hash functions and pairing. Clearly, the computation efficiency of our scheme is notably superior to that of the other four methods.

Fig. 3 shows simulation results for varying number of challenge files ($n = 20, 40, 60, 80, 100, 120$). Since the **ChalGen** algorithm in our scheme is automated by blockchain, parameters passing between Solidity smart contracts and the Node.js environment involve multiple type transformations. This makes our approach less efficient compared to other schemes in **ChalGen**. As illustrated in Fig. 4, the total time cost during an integrity auditing phase (including the algorithms **ChalGen**, **ProofGen** and **ProofVerify**) in all schemes increases linearly with the number of file blocks and our scheme is the lowest when compared to other schemes. For example, to audit 25 file blocks, the total time cost of our scheme is approximately 0.29 s, which is about 14.43% of the 2.01 s in [17], 28.43% of the 1.02 s in [18], 29.59% of the 0.98 s in [21], and 51.79% of the 0.56 s in [22], 93.55% of the 0.31 s in [26], and 20.57% of the 1.41 s in [27], respectively. As shown in Fig. 4, it can be observed that the time cost of a complete auditing phase is linearly related to the number of file blocks. In comparison, the proposed method demonstrates superior efficiency and practical applicability.

2) *Communication Cost Comparison*: This subsection compares these schemes in terms of the verifiable tag length and the auditing proof length. According to the data presented in Table V, the communication expenses incurred by our scheme are comparatively reduced in comparison to those of the existing schemes. In the table, The variable r represents the number of sectors into which each file block is divided. In our scheme, $r = 1$, meaning each file block is not further divided into multiple sectors; instead, it is treated as a single sector. Specifically, the length of the verifiable labels for each data block in our scheme is reduced by 32 bits compared to most other schemes. Although our provable length is not optimal, its size remains constant in magnitude.

B. Blockchain Experiments

In order to test our LB-CL-DIA scheme and the smart contracts, we developed a simulation experiment based on the Ethereum blockchain platform. [48] We utilized the Hardhat framework in Node.js to set up a local Ethereum environment, which includes cloud, user, EdgeNode entities and their associated algorithms written in JavaScript, and smart contracts written in Solidity. For performing arithmetic, computations, and elliptic curve group operations in the $\mathbb{Z}_q^*$, we leveraged the BIG and ECP data type from the amcl library.

As illustrated in Table VI, we evaluate the efficiency of our experiments based on execution time, gas fee, and communication overhead. We set the number of file blocks to 100. Firstly, Blockchain execution time includes the blockchain call time (the time required to mine a transaction into the blockchain), and the time taken for the smart contract to perform specific operations (such as proof verification), and the time taken to successfully retrieve the return value from the smart contract. These blockchain call times are independent of the file size. Consequently, the variations in overall execution time arise from the differing computation time of the parties involved, which vary according to the number of file blocks. Secondly, our blockchain is deployed in a local environment, and the gas fees mentioned are virtual costs provided by

TABLE IV
COMPUTATION COST COMPARISON

Scheme	TagGen	ChalGen	ProofGen	ProofVerify
[17]	$n(T_m + 2T_e + T_{mtp})$	$3T_r$	$0.1nT_m + 0.1nT_e$	$(0.1n + 2)T_p + 0.3nT_m + 0.2nT_e + 0.2nT_{mtp}$
[18]	$n(2T_m + 3T_e + T_{mtp} + T_r)$	$0.1nT_r$	$0.1nT_m + 0.1nT_e$	$5T_p + 0.2nT_m + (0.12n + 1)T_e + (0.1n + 2)T_{mtp}$
[21]	$n(2T_m + 2T_e + T_{mtp})$	$0.1nT_r$	$0.1nT_m + 0.1nT_e$	$3T_p + (0.1n + 1)T_m + (0.1n + 2)T_e + (0.1n + 1)T_{mtp} + T_c$
[22]	$n(2T_m + 3T_e + T_{mtp} + T_r)$	$0.2nT_r$	$0.2T_m + 0.1nT_e$	$3T_p + 0.2nT_m + T_e + 0.1nT_{mtp} + T_c$
[26]	$n(3T_h + T_{pm}) + (n + 1)T_r$	$3T_r$	$0.3n(T_h + T_{pm}) + 0.1nT_{pa}$	$0.3nT_h + 2T_{pa} + (2 + 0.1n)T_{pm}$
[27]	$(T_h + T_m + 2T_e)$	$0.5nT_h$	$0.5nT_h + 0.1nT_e$	$0.2n(T_e + T_m + T_h) + T_h + T_e + 3T_p$
Ours	$2nT_h + T_{pm} + (n + 1)T_r$	$0.2nT_{kh}$	$0.1n(T_{pa} + T_{pm} + T_h) + T_r + T_{pm}$	$2T_{pa} + 4T_{pm} + (0.1n + 1)T_h$

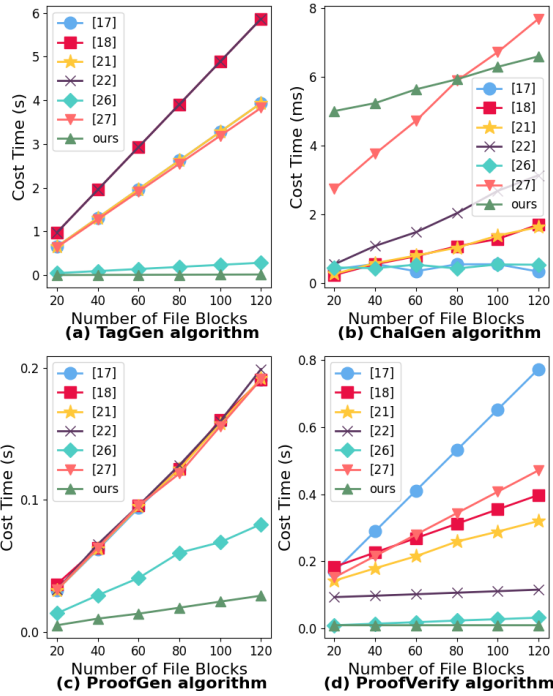


Fig. 3. Cost time of different algorithms versus the number of challenge file blocks.

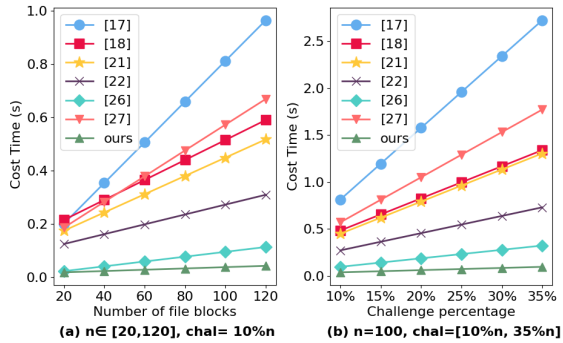


Fig. 4. Total cost time of an auditing phase versus the number of challenge file blocks.

Hardhat. In **StoreRes**, the primary gas fee arises from storage operations. In **ChanGen**, the gas fee is primarily driven by random selection and storage. The current linear deduplication approach and repeated hash computations result in

TABLE V
COMMUNICATION COST COMPARISON

Scheme	Tag length	Proof length
[17]	$ \mathbb{G}_1 $	$r \mathbb{G}_1 + r \mathbb{Z}_q^* $
[18]	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + r \mathbb{Z}_q^* $
[21]	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + \mathbb{Z}_q^* $
[22]	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + r \mathbb{Z}_q^* $
[26]	$ \mathbb{G} + \mathbb{Z}_q^* $	$2 \mathbb{G} + \mathbb{Z}_q^* $
[27]	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + r \mathbb{Z}_q^* $
Ours	$ \mathbb{G} + \mathbb{Z}_q^* $	$2 \mathbb{G} + 2 \mathbb{Z}_q^* $

$|\mathbb{G}_1| = 512 \text{ bits}$, $|\mathbb{Z}_q^*| = 160 \text{ bits}$, and $|\mathbb{G}| = 320 \text{ bits}$.

TABLE VI
DATA RELATED TO DIFFERENT SMART CONTRACT OPERATIONS

Smart contract	Call Time (sec)	Execution Time (sec)	Gas fee (Wei)	Communication overhead (bytes)
StoreRes	0.002	0.150	32455	64
ChalRes	0.002	0.014	64744	39
ProVerify	0.002	0.338	8397944	525

significant gas cost. In **ProofVerify**, the gas fee is dominated by elliptic curve operations, specifically scalar multiplication and point addition. Although off-chain computation could mitigate these costs, our evaluation in a local environment focuses on verifying the correctness and feasibility of the scheme. Optimization strategies, including off-chain execution, are deferred to future work. Furthermore, since our experiment is implemented using Node.js and Solidity, there are numerous issues with the conversion of data types between these two languages. To avoid the “stack too deep” error encountered during the operation of our local blockchain, all parameters are encapsulated within an array of the uint256 data type. Lastly, in this local setup, all nodes run on the same host with negligible communication latency, and Hardhat’s auto-mining ensures near instant transaction confirmation. Thus, end-to-end latency is not evaluated as a separate metric. In future work, we will measure and analyze end-to-end latency in distributed or real-world blockchain environments to evaluate system performance under realistic conditions.

Fig. 5 illustrates the execution time for three smart contracts **StoreRes**, **ChalRes** and **ProVerify** as the number of file blocks

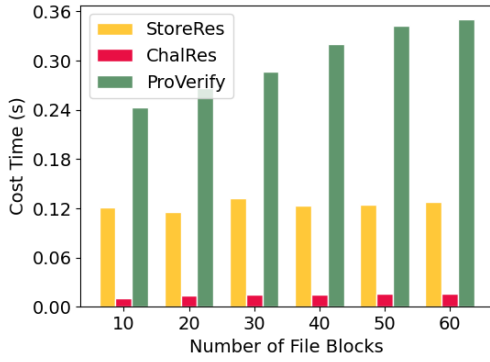


Fig. 5. Execution time of different smart contracts versus the number of challenge file blocks.

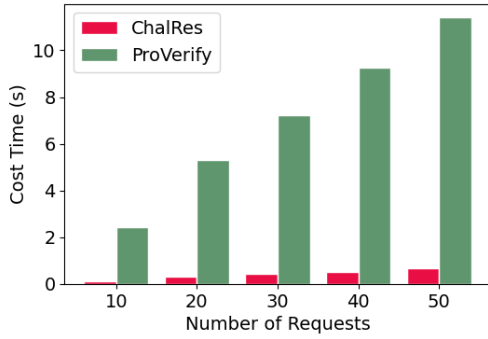


Fig. 6. Computation costs of different smart contracts related to the number of requests.

varies. Firstly, the elements stored in the smart contract **StoreRes** are independent of the number of file blocks, resulting in a relatively stable and flat overall trend. Secondly, in the smart contract **ChalRes**, the smart contract is designed to select 10% of the total file blocks for verification. Within the smart contract, a hash function is employed to generate random numbers, which incurs significantly higher computation costs compared to operations in the other schemes, as also depicted in Fig. 3(b). Lastly, the smart contract **ProVerify** involves performing scalar multiplication on elliptic curves and point addition, both of which are computationally intensive algorithms. Please note that the use of hash functions for generating randomness within blockchain environments can be costly due to their inherent complexity, especially when involving elliptic curve cryptography. These operations are crucial for ensuring the security and integrity of the transactions but lead to increased computation overhead.

Fig. 6 illustrates the execution time for the smart contracts **ChalRes** and **ProVerify** when an edge node makes multiple data integrity auditing requests to the blockchain, where the number of challenge file blocks is set to 50. The computation costs of both smart contracts depend on the size of the file blocks, and experimental results confirm that time consumption is roughly linear with the number of requests. This is attributed to the use of an asynchronous function, which allows requests to be processed sequentially and effectively avoids collisions. Such a mechanism ensures that even when receiving multiple data integrity auditing requests, the system can efficiently process and respond.

VII. CONCLUSION

In this paper, we put forward a lightweight blockchain-assisted CL-DIA scheme without TPA. To avoid the shortcomings (such as performance bottleneck and single-point failure) caused by the TPA-based centralized auditing model in the existing CL-DIA schemes, we introduce a blockchain-assisted auditing model. Specifically, we utilize the blockchain network instead of a single TPA for DIA. We employ smart contracts to generate challenge messages and verify the auditing proofs, thereby addressing the problems inherent in the current TPA-based CL-DIA schemes. We show formal security proofs of our scheme relying on the hardness assumption of the ECDLP in the random oracle model. As far as we know, it is the sole lightweight CL-DIA scheme to date that concurrently satisfies the unforgeability of verifiable tags and auditing proofs. Finally, theoretical analysis and experimental evaluations show that our scheme has notable performance advantages against current CL-DIA schemes.

We plan to extend this work in three directions to better align with practical requirements: First, while our current security proof relies on the standard ECDLP assumption, future work will formally model the trustworthiness of smart contract execution, potentially using frameworks like Universal Composability (UC). Second, our current implementation uses a localized blockchain network. Exploring deployment on public blockchains and optimizing underlying algorithms (particularly to reduce gas consumption) remains a key research direction. Third, this study primarily addresses integrity auditing in centralized cloud storage. Decentralized storage systems like InterPlanetary File System (IPFS) offer a highly promising alternative. IPFS employs content addressing, where each file is identified by a unique Content Identifier (CID) derived from its cryptographic hash. By storing data on IPFS and recording its CID as immutable metadata on the blockchain, a robust integrity verification mechanism is established: any data modification alters the CID, making tampering immediately detectable. This “blockchain-IPFS” hybrid architecture provides a more resilient and transparent foundation for secure data storage and represents a significant avenue for future research.

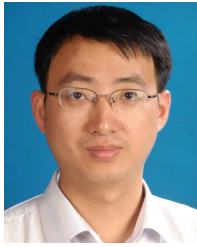
REFERENCES

- [1] C. Shi, Q. Huang, R. Jian, and G. Chi, “Cross-domain inner-product access control encryption for secure EMR flow in cloud edge,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9866–9880, 2024.
- [2] G. Ateniese et al., “Provable data possession at untrusted stores,” in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.
- [3] C. C. Erway, A. Küpçü, C. Papamanthou, and R. Tamassia, “Dynamic provable data possession,” *ACM Trans. Inf. Syst. Secur.*, vol. 17, no. 4, pp. 213–222, 2009.
- [4] Q. Wang, C. Wang, K. Ren, W. Lou, and J. Li, “Enabling public auditability and data dynamics for storage security in cloud computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 5, pp. 847–859, May 2011.
- [5] C. Wang, Q. Wang, K. Ren, and W. Lou, “Privacy-preserving public auditing for data storage security in cloud computing,” in *Proc. IEEE INFOCOM*, Mar. 2010, pp. 1–9.
- [6] J. Yu, K. Ren, and C. Wang, “Enabling cloud storage auditing with verifiable outsourcing of key updates,” *IEEE Trans. Inf. Forensics Security*, vol. 11, no. 6, pp. 1362–1375, Jun. 2016.
- [7] B. Wang, B. Li, and H. Li, “Panda: Public auditing for shared data with efficient user revocation in the cloud,” *IEEE Trans. Services Comput.*, vol. 8, no. 1, pp. 92–106, Jan. 2015.

- [8] M. Thangavel and P. Varalakshmi, "Enabling ternary hash tree based integrity verification for secure cloud data storage," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 12, pp. 2351–2362, Dec. 2020.
- [9] Z. Hao, S. Zhong, and N. Yu, "A privacy-preserving remote data integrity checking protocol with data dynamics and public verifiability," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 9, pp. 1432–1437, Sep. 2011.
- [10] T. Wu, G. Yang, Y. Mu, F. Guo, and R. H. Deng, "Privacy-preserving proof of storage for the pay-as-you-go business model," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 2, pp. 563–575, Mar. 2021.
- [11] J. Yuan and S. Yu, "Efficient public integrity checking for cloud data sharing with multi-user modification," in *Proc. IEEE Conf. Comput. Commun.*, May 2014, pp. 2121–2129.
- [12] B. Wang, B. Li, and H. Li, "Oruta: Privacy-preserving public auditing for shared data in the cloud," *IEEE Trans. Cloud Comput.*, vol. 2, no. 1, pp. 43–56, Jan. 2014.
- [13] Y. Li, Y. Yu, G. Min, W. Susilo, J. Ni, and K.-K.-R. Choo, "Fuzzy identity-based data integrity auditing for reliable cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 16, no. 1, pp. 72–83, Jan. 2019.
- [14] W. Shen, J. Qin, J. Yu, R. Hao, and J. Hu, "Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 2, pp. 331–346, Feb. 2019.
- [15] S. S. Al-Riyami and K. G. Paterson, "Certificateless public key cryptography," in *Advances in Cryptology—ASIACRYPT 2003*. Cham, Switzerland: Springer, 2003, pp. 452–473.
- [16] Y. Ji, B. Shao, J. Chang, and G. Bian, "Privacy-preserving certificateless provable data possession scheme for big data storage on cloud, revisited," *Appl. Math. Comput.*, vol. 386, Dec. 2020, Art. no. 125478.
- [17] J. Li, H. Yan, and Y. Zhang, "Certificateless public integrity checking of group shared data on cloud storage," *IEEE Trans. Services Comput.*, vol. 14, no. 1, pp. 71–81, Jan. 2021.
- [18] L. Zhou, A. Fu, G. Yang, H. Wang, and Y. Zhang, "Efficient certificateless multi-copy integrity auditing scheme supporting data dynamics," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1118–1132, Mar. 2022.
- [19] B. Wang, B. Li, H. Li, and F. Li, "Certificateless public auditing for data integrity in the cloud," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Oct. 2013, pp. 136–144.
- [20] X. Zhang, X. Wang, D. Gu, J. Xue, and W. Tang, "Conditional anonymous certificateless public auditing scheme supporting data dynamics for cloud storage systems," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 4, pp. 5333–5347, Dec. 2022.
- [21] Y. Huang, W. Shen, and J. Qin, "Certificateless cloud storage auditing supporting data ownership transfer," *Comput. Secur.*, vol. 139, Apr. 2024, Art. no. 103738.
- [22] J. Shen, P. Zeng, K.-K.-R. Choo, and C. Li, "A certificateless provable data possession scheme for cloud-based EHRs," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1156–1168, 2023.
- [23] G. Yang, L. Han, J. Bi, and F. Wang, "A collusion-resistant certificateless provable data possession scheme for shared data with user revocation," *Cluster Comput.*, vol. 27, no. 2, pp. 2165–2179, Apr. 2024.
- [24] Y. Huang, W. Shen, J. Qin, and H. Hou, "Privacy-preserving certificateless public auditing supporting different auditing frequencies," *Comput. Secur.*, vol. 128, May 2023, Art. no. 103181.
- [25] P. Banerjee, N. Nikam, S. Mazumdar, and S. Ruj, "Cumulus: Blockchain-enabled privacy-preserving data audit in cloud," *Distrib. Ledger Technol., Res. Pract.*, vol. 4, no. 3, pp. 1–28, Sep. 2025.
- [26] J. Zhao, Y. Zheng, H. Huang, J. Wang, X. Zhang, and D. He, "Lightweight certificateless privacy-preserving integrity verification with conditional anonymity for cloud-assisted medical cyber-physical systems," *J. Syst. Archit.*, vol. 138, Apr. 2023, Art. no. 102860.
- [27] Y. Miao, K. Gai, J. Yu, Y.-A. Tan, L. Zhu, and W. Meng, "Blockchain-empowered keyword searchable provable data possession for large similar data," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1374–1389, 2025.
- [28] Y. Deswarte, J.-J. Quisquater, and A. Saïdane, "Remote integrity checking: How to trust files stored on untrusted servers," in *Proc. Work. Conf. Integrity Internal Control Inf. Syst.*, May 2004, pp. 1–11.
- [29] H. Yan, J. Li, J. Han, and Y. Zhang, "A novel efficient remote data possession checking protocol in cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 1, pp. 78–88, Jan. 2017.
- [30] H. Yan, J. Li, and Y. Zhang, "Remote data checking with a designated verifier in cloud storage," *IEEE Syst. J.*, vol. 14, no. 2, pp. 1788–1797, Jun. 2020.
- [31] J. Li, H. Yan, and Y. Zhang, "Efficient identity-based provable multi-copy data possession in multi-cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 1, pp. 356–365, Jan. 2022.
- [32] S. Li, C. Xu, Y. Zhang, Y. Du, and K. Chen, "Blockchain-based transparent integrity auditing and encrypted deduplication for cloud storage," *IEEE Trans. Services Comput.*, vol. 16, no. 1, pp. 134–146, Jan. 2023.
- [33] J. Xue, C. Xu, J. Zhao, and J. Ma, "Identity-based public auditing for cloud storage systems against malicious auditors via blockchain," *Sci. China Inf. Sci.*, vol. 62, no. 3, pp. 1–16, Mar. 2019.
- [34] Y. Zhang, C. Xu, X. Lin, and X. Shen, "Blockchain-based public integrity verification for cloud storage against procrastinating auditors," *IEEE Trans. Cloud Comput.*, vol. 9, no. 3, pp. 923–937, Jul. 2021.
- [35] Y. Lin, J. Li, S. Kimura, Y. Yang, Y. Ji, and Y. Cao, "Consortium blockchain-based public integrity verification in cloud storage for IoT," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3978–3987, Mar. 2022.
- [36] H. Wang, H. Qin, M. Zhao, X. Wei, H. Shen, and W. Susilo, "Blockchain-based fair payment smart contract for public cloud storage auditing," *Inf. Sci.*, vol. 519, pp. 348–362, May 2020.
- [37] J. Sengupta, S. Ruj, and S. D. Bit, "FairShare: Blockchain enabled fair, accountable and secure data sharing for industrial IoT," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2929–2941, Sep. 2023.
- [38] H. Yan, Y. Liu, S. Qiu, S. Hu, W. Zhang, and J. Xia, "Towards public integrity audit for cloud-IoT data based on blockchain," *Comput. Syst. Sci. Eng.*, vol. 41, no. 3, pp. 1129–1142, 2022.
- [39] Q. Zhang, D. Sui, J. Cui, C. Gu, and H. Zhong, "Efficient integrity auditing mechanism with secure deduplication for blockchain storage," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2365–2376, Aug. 2023.
- [40] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4928–4945, Nov. 2023.
- [41] J. Du, G. Dong, J. Ning, Z. Xu, and R. Yang, "A blockchain-assisted certificateless public cloud data integrity auditing scheme," *IEEE Access*, vol. 11, pp. 123018–123029, 2023.
- [42] Q. Liu, X. Zhang, J. Xue, R. Zhou, X. Wang, and W. Tang, "Enabling blockchain-assisted certificateless public integrity checking for industrial cloud storage systems," *J. Syst. Archit.*, vol. 140, Jul. 2023, Art. no. 102898.
- [43] L. Wang, Z. Guan, Z. Chen, and M. Hu, "Enabling integrity and compliance auditing in blockchain-based gdpr-compliant data management," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 20955–20968, Aug. 2023.
- [44] Y. Yu et al., "Identity-based remote data integrity checking with perfect data privacy preserving for cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 4, pp. 767–778, Apr. 2017.
- [45] J. Ni, K. Zhang, Y. Yu, and T. Yang, "Identity-based provable data possession from RSA assumption for secure cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 3, pp. 1753–1769, May 2022.
- [46] D. Pointcheval and J. Stern, "Security proofs for signature schemes," in *Proc. Int. Conf. Theory Appl. Cryptograph. Techn.* Cham, Switzerland: Springer, May 1996, pp. 387–398.
- [47] J. A. Akinyele et al., "Charm: A framework for rapidly prototyping cryptosystems," *J. Cryptograph. Eng.*, vol. 3, no. 2, pp. 111–128, Jun. 2013.
- [48] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, vol. 151, pp. 1–32, Jul. 2014.



Chen Zhu received the B.S. degree in mathematics and the M.S. degree in electronic and information engineering from Nanjing Normal University, Nanjing, China, in 2021 and 2025, respectively. Her research interests include information security and authenticated key exchange.



Yang Lu received the B.S. degree in mathematics and the M.S. degree in computer science from Nanjing Normal University, Nanjing, China, in 2000 and 2003, respectively, and the Ph.D. degree in computer science from the PLA University of Science and Technology, Nanjing, in 2009. Currently, he is a Professor with the School of Computer and Electronic Information, Nanjing Normal University. His research interests include information security and cryptography.



Jiguo Li received the B.S. degree in mathematics from Heilongjiang University, Harbin, China in 1996, and the M.S. degree in mathematics and the Ph.D. degree in computer science from Harbin Institute of Technology, Harbin, in 2000 and 2003, respectively. He is currently a professor with the College of Computer and Cyber Security, Fujian Normal University, Fuzhou, China. His research interests include cryptography and information security, cloud computing, and wireless security.



Nian Xia (Member, IEEE) received the B.S. degree in electrical engineering from Northeastern University, Shenyang, China, in 2012, and the M.S. and Ph.D. degrees from the Institute of Computer and Communication Engineering, Department of Electrical Engineering, National Cheng Kung University, Tainan, Taiwan, in 2014 and 2020, respectively. He is currently a Lecturer with the School of Computer and Electronic Information, Nanjing Normal University, China. His research interests include wireless communications and network security.



Yinxia Sun received the B.S. and M.S. degrees in mathematics from Nanjing Normal University, Nanjing, China, in 2003 and 2008, respectively, and the Ph.D. degree in cryptography from Xidian University, Xi'an, China, in 2011. She is currently an Associate Professor with the School of Computer and Electronic Information, Nanjing Normal University. Her research interests include public key encryption, digital signature and their applications in IoT, cloud storage, and blockchain.